GEMPRO Notebook - Extended

GEMPRO

Índice

1 Contest **3**
1.1 template.cpp . . . 3
1.2 .bashrc . . . 3
1.3 Template de stress testing . . . 3
1.4 .vimrc . . . 4

2 Matemática **4**
2.1 Potência módulo `mod` . . . 4
2.2 Raiz quadrada módulo `mod` . . . 4
2.3 Algoritmo Euclideano . . . 4
2.4 Soma de Floor . . . 5
2.5 Crivo . . . 5
2.6 CRT . . . 5
2.7 Base de XOR . . . 6
2.8 Teste de primalidade . . . 6
2.9 Fatoração de inteiros . . . 6

3 Estruturas de Dados **7**
3.1 Fenwick Tree . . . 7
3.2 Segtree . . . 7
3.3 Lazy Segtree . . . 7
3.4 Sparse Table . . . 8
3.5 Disjoint Set Union . . . 8
3.6 Convolução . . . 9
3.6.1 FFT . . . 9
3.6.2 NTT . . . 9
3.7 Convolução de XOR . . . 10
3.8 Max Add Segtree . . . 10
3.9 Zeta e Mobius . . . 11
3.10 Árvore de Li Chao . . . 11
3.11 Árvore de Li Chao++ . . . 12
3.12 Árvore Cartesiana . . . 12
3.13 Árvore de Busca Binária Aleatória . . . 13
3.14 Treap implícita . . . 13
3.15 Treap de conjunto ordenado . . . 14

4 Grafos **15**
4.1 Dijkstra . . . 15
4.2 0-1 BFS . . . 16
4.3 Caminho Euleriano . . . 16
4.4 Árvore de pontes . . . 16
4.5 Árvore de bloco e corte . . . 17
4.6 Dominator Tree . . . 17
4.7 Árvore geradora mínima direcionada . . . 18
4.7.1 Encontrando qualquer caminho euleriano . . . 18
4.8 Floresta de grafo funcional . . . 18
4.9 Componentes fortemente conexas e grafo de condensação . . . 19
4.10 2-SAT . . . 19
4.11 Matching bipartido máximo . . . 20
4.12 Max Flow . . . 20
4.12.1 Dinic . . . 20
4.13 Min Cost Flow . . . 21
4.13.1 MCMF . . . 21
4.14 Matching bipartido com menor custo . . . 22

5 Árvores **22**
5.1 Base . . . 22
5.2 Utilidades . . . 23
5.3 LCA . . . 23
5.4 Matching em Árvores . . . 24

6 Strings **24**
6.1 KMP . . . 24
6.2 Z-function . . . 24
6.3 Array de sufixos . . . 25
6.4 Maior prefixo comum (LCP) . . . 25

6.5 Árvore de sufixos 25
6.6 Automato de sufixos 26
6.7 Encontrando os palíndromos máximos em uma string 26
6.8 Aho-Corasick 27

7 Geometria **27**
7.1 Base 27
7.2 Interseção de retas 28
7.3 Interseção de segmentos 28
7.4 Área de um polígono 28
7.5 Verificar se um ponto está dentro de um polígono 28
7.6 Convex hull 28
7.7 Menor distância euclideana 29
7.8 Interseção de círculo e reta 29
7.9 Interseção de círculos 30
7.10 Tangentes comuns a dois círculos 30
7.11 Soma Minkowski de polígonos convexos 30
7.12 Corte de Polígono 30

A Fórmulas, Teoremas e Fatos **31**
A.1 Combinatória e Sequências 31
A.2 Teoria dos Números 32
A.3 Grafos e Jogos 33
A.4 Intervalos, Prefixos e Contribuições 34
A.5 Strings 34
A.6 Geometria 35
A.7 DP e Otimizações 35

B Ideias **36**
B.1 Gerais 36
B.2 Calculando somas e integrais 36
B.3 Problemas de Otimização 36

C Debugging **36**
C.1 Geral 36
C.2 WA - Wrong Answer 36
C.3 RE - Runtime Error 36

Contest

template.cpp

Template base para uma solução de contest. Interface: escreva a lógica do problema no próprio arquivo e reaproveite aliases, macros e utilitários já definidos, em vez de montar boilerplate a cada questão.

```cpp
#include <bits/stdc++.h>

using namespace std;

using ll = long long;
using ld = long double;
using pii = pair<int, int>;
using pll = pair<ll, ll>;
using vi = vector<int>;

#define pb push_back
#define eb emplace_back
#define fi first
#define se second
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
#define rep(i,a,b) for (int i = (a); i < (b); ++i)

mt19937 rng(random_device{}());

int main() {
    cin.tie(0)->sync_with_stdio(0);
}
```

.bashrc

Atalhos e funções de shell para o ambiente de prova. Interface: carregue o arquivo no shell e use os comandos definidos para compilar, rodar casos de teste e fazer diff das saídas.

```bash
c() { g++ $1.cpp -o $1 -std=c++20; }
cs() {
    g++ $1.cpp -o $1 -std=c++20 \
        -fsanitize=address,undefined \
        -fsanitize=signed-integer-overflow -ggdb
}
gen() {
    local N=${1:-10}
    for i in $(seq 1 $N); do
        python3 gen.py > $i.in
        if [[ -f brute ]]; then
            ./brute < $i.in > $i.ans
        fi
    done
}
rr() {
    for i in $(ls *.in); do
        local f=$(basename $i .in)
        ./$1 < $f.in > $f.out
        if [[ -f $f.ans ]]; then
            if ! cmp -s $f.out $f.ans; then
                echo WA
                return 1
            fi
        fi
    done
    echo AC
}
```

Template de stress testing

Gerador base para stress testing local. Interface: complete a geração dos casos e rode o script junto com um brute e uma solução candidata até achar um contraexemplo.

```python
from random import randrange as rr, randint as ri, shuffle as sh, sample as sm
from string import ascii_lowercase as alpha
from itertools import combinations as combs

def out(*a): print(" ".join(map(str, a)))

def outa(a): print(" ".join(map(str, a)))

def tree(n): return [[rr(1, i), i] for i in range(2, n + 1)]

def graph(n, m): return sm(list(combs(range(1, n + 1), 2)), m)

def arr(n, l = 0, r = 10 ** 9): return [ri(l, r) for i in range(n)]
```

```
def darr(n, l = 0, r = 10 ** 9): return sm(range(l, r), n)

def perm(n): p = list(range(1, n + 1)); sh(p); return p
```

.vimrc

Configuração do Vim voltada para edição rápida em prova. Interface: use como vimrc local para obter opções e atalhos pensados para codar sob tempo.

```
syntax on
set cin ts=4 sw=4 udf cul nu is

ca Hash w !cpp -dD -P -fpreprocessed \| tr -d '[:space:]' \| md5sum \| cut -c -6
```

Matemática

Potência módulo `mod`

Exponenciação binária módulo constante. Interface: ajuste a constante `mod` no próprio snippet e chame `powm(x, e)` para calcular $x^e \bmod mod$ com `e >= 0`. Útil em combinações, hashes e inverso modular quando o módulo permite. Complexidade: $O(\log e)$.

44bf2f - math/powm.cpp - 10 lines

```
const ll mod = 998244353; // Change this
ll powm(ll x, ll e) {
    ll r = 1;
    while (e) {
        if (e & 1) (r *= x) %= mod;
        (x *= x) %= mod;
        e >>= 1;
    }
    return r;
} // e2de55
```

Raiz quadrada módulo `mod`

Tonelli–Shanks para módulo primo. Interface: ajuste a variável global `mod` para um primo ímpar e chame `sqrtMod(a)`. O retorno é a menor raiz não negativa de `x² % mod = a`; se não existir raiz, a função retorna `-1`. Os casos `a = 0` e `mod = 2` já são tratados. Complexidade: $O(\log^2 mod)$ multiplicações modulares.

b04743 - math/mod-sqrt.cpp - 19 lines

```
int sqrtMod(int a) { // mod prime
    if (a == 0 || mod == 2) return a;
    if (powm(a, (mod - 1) / 2) != 1) return -1;
    if (mod % 4 == 3) return powm(a, (mod + 1) / 4);
    int s = __builtin_ctz(mod - 1), q = (mod - 1) >> s, z = 2;
    while (powm(z, (mod - 1) / 2) != mod - 1) z++;
    ll c = powm(z, q), x = powm(a, (q + 1) / 2), t = powm(a, q);
    for (int m = s; t != 1; ) {
        ll tt = t;
        int i = 0;
        while (tt != 1) tt = tt * tt % mod, i++;
        ll b = powm(c, 1 << (m - i - 1));
        x = x * b % mod;
        c = b * b % mod;
        t = t * c % mod;
        m = i;
    }
    return min<ll>(x, mod - x);
} // b04743
```

Algoritmo Euclideano

Algoritmo de Euclides estendido. Interface: chame `egcd(a, b, x, y)`; o retorno é $g = gcd(a, b)$ e as referências `x`, `y` saem com $ax + by = g$. Útil para inverso modular e CRT. Complexidade: $O(\log(\min(a, b)))$.

b4f64d - math/egcd.cpp - 9 lines

```
ll egcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) {
        x = 1, y = 0;
        return a;
    }
    ll g = egcd(b, a % b, y, x);
    y -= (a / b) * x;
    return g;
} // b4f64d
```

Soma de Floor

Rotinas do tipo Euclides para somas com piso. Interface: `floorSum(n, m, a, b)` devolve $\sum_{i=0}^{n-1} \lfloor (ai+b)/m \rfloor$, `floorSumI` devolve $\sum i \lfloor (ai+b)/m \rfloor$ e `floorSumSq` devolve $\sum \lfloor (ai+b)/m \rfloor^2$. Todas retornam `i128`; faça cast fora do snippet se precisar imprimir. Complexidade: $O(\log(\max(a,b,m)))$ por chamada.

84ad33 - math/floor-sum.cpp - 45 lines

```
using i128 = __int128_t;

i128 floorSumSq(ll n, ll m, ll a, ll b);

i128 floorSum(ll n, ll m, ll a, ll b) {
    if (!n) return 0;
    if (a >= m || b >= m) {
        ll qa = a / m, qb = b / m;
        return (i128)qa * n * (n - 1) / 2 + (i128)qb * n
            + floorSum(n, m, a % m, b % m);
    }
    ll k = ((i128)a * (n - 1) + b) / m;
    if (!k) return 0;
    return (i128)k * (n - 1) - floorSum(k, a, m, m - b - 1);
} // 440661

i128 floorSumI(ll n, ll m, ll a, ll b) {
    if (!n) return 0;
    i128 s1 = (i128)n * (n - 1) / 2, s2 = (i128)n * (n - 1) * (2 * n - 1) / 6;
    if (a >= m || b >= m) {
        ll qa = a / m, qb = b / m;
        return (i128)qa * s2 + (i128)qb * s1
            + floorSumI(n, m, a % m, b % m);
    }
    ll k = ((i128)a * (n - 1) + b) / m;
    if (!k) return 0;
    return (i128)k * s1
        - (floorSumSq(k, a, m, m - b - 1) + floorSum(k, a, m, m - b - 1)) / 2;
} // dd026c
i128 floorSumSq(ll n, ll m, ll a, ll b) {
    if (!n) return 0;
    i128 s1 = (i128)n * (n - 1) / 2, s2 = (i128)n * (n - 1) * (2 * n - 1) / 6;
    if (a >= m || b >= m) {
        ll qa = a / m, qb = b / m;
        return (i128)qa * qa * s2 + 2 * (i128)qa * qb * s1 + (i128)qb * qb * n
            + 2 * (i128)qa * floorSumI(n, m, a % m, b % m)
            + 2 * (i128)qb * floorSum(n, m, a % m, b % m)
            + floorSumSq(n, m, a % m, b % m);
    }
    ll k = ((i128)a * (n - 1) + b) / m;
    if (!k) return 0;
    return (i128)k * k * (n - 1)
        - 2 * floorSumI(k, a, m, m - b - 1)
        - floorSum(k, a, m, m - b - 1);
} // eb1940
```

Crivo

Crivo linear. Interface: construa `Sieve sv(n)`; `sv.primes` contém todos os primos menores que n e `sv.spf[x]` guarda o menor fator primo de `x`. Complexidade: $O(n)$.

eb3eff - math/sieve.cpp - 16 lines

```
struct Sieve {
    vi primes, spf;
    Sieve(int n): spf(n) {
        rep (i, 2, n) {
            if (!spf[i]) {
                spf[i] = i;
                primes.push_back(i);
            }
            for (int j: primes) {
                if (j * i >= n) break;
                spf[i * j] = j;
                if (j == spf[i]) break;
            }
        }
    }
};
```

CRT

Merge de duas congruências. Interface: `crt(r1, m1, r2, m2)` retorna `{r, m}` para $x \equiv r \pmod{m}$, ou `{0, -1}` se o sistema for incompatível. Chame em cadeia para combinar mais congruências. Requer `egcd`. Complexidade: $O(\log(\min(m_1, m_2)))$.

6e0e1c - math/crt.cpp - 12 lines

```
// x = r1 (mod m1), x = r2 (mod m2) -> x = r (mod m)
// returns {r, m} or {0, -1} if incompatible
```

```
// chain calls to combine more congruences
pair<ll, ll> crt(ll r1, ll m1, ll r2, ll m2) {
    ll x, y, g = egcd(m1, m2, x, y);
    if ((r2 - r1) % g) return {0, -1};
    ll k = m2 / g, m = m1 / g * m2;
    ll q = ((r2 - r1) / g % k + k) % k;
    x = ((x % k) + k) % k;
    ll r = r1 + (__int128)m1 * (q * x % k);
    return {(r % m + m) % m, m};
} // 6e0e1c
```

Base de XOR

Base linear para XOR em `ll` não negativos. Interface: construa `XorBasis xb`; use `add(x)` para inserir um valor, `minXor(x)` para minimizar `x` aplicando XOR com os vetores da base, e `maxXor(x)` para maximizar `x`. Em particular, `minXor(x) == 0` sse `x` é combinação XOR dos elementos inseridos. Complexidade: $O(B)$ por operação, com $B = 63$.

0e002d - math/xor-basis.cpp - 20 lines

```
struct XorBasis {
    static const int B = 63; // works for nonnegative ll values < 2^63
    ll b[B] = {};
    bool add(ll x) {
        for (int i = B - 1; i >= 0; i--) if (x >> i & 1) {
            if (!b[i]) return b[i] = x, 1;
            x ^= b[i];
        }
        return 0;
    } // 9dfd7d

    ll minXor(ll x) {
        for (int i = B - 1; i >= 0; i--) x = min(x, x ^ b[i]);
        return x;
    } // f6abc2
    ll maxXor(ll x = 0) {
        for (int i = B - 1; i >= 0; i--) x = max(x, x ^ b[i]);
        return x;
    } // e00cc1
};
```

Teste de primalidade

Miller–Rabin determinístico para `ll`. Interface: chame `isPrime(n)`; o retorno é booleano e funciona para inteiros de 64 bits. Complexidade: $O(\log n)$ com constante pequena.

b5a468 - math/primality.cpp - 25 lines

```
using i128 = __int128;

bool checkPrime(ll p, ll a) {
    ll k = p - 1, d = 1;
    while (~k & 1) k >>= 1;
    for (ll e = k; e; e >>= 1) {
        if (e & 1) d = i128(d) * a % p;
        a = i128(a) * a % p;
    }
    if (d == 1 || d == p - 1) return true;
    while ((k <<= 1) < p - 1) {
        d = i128(d) * d % p;
        if (d == p - 1) return true;
    }
    return false;
} // d5cf5e

bool isPrime(ll p) {
    if (p == 1) return false;
    for (int i: {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
        if (p == i) return true;
        if (!checkPrime(p, i)) return false;
    }
    return true;
} // a137c8
```

Fatoração de inteiros

Pollard–Rho para fatoração de inteiros de 64 bits. Interface: chame `factor(n, fs)` com `n > 0`; a função adiciona em `fs` os fatores primos de `n`, com repetição e sem ordenar. Se precisar da fatoração comprimida, ordene e agrupe fora do snippet. O template usa `isPrime(n)` do mesmo arquivo. Complexidade heurística: sublinear em `n` por fator encontrado, sem pior caso simples útil para contest.

cff773 - math/factor.cpp - 28 lines

```
mt19937_64 rng64(random_device{}());

using ull = unsigned long long;
using u128 = __uint128_t;

ull modMul(ull a, ull b, ull mod) { return (u128)a * b % mod; }

ull rho(ull n) {
    if (n % 2 == 0) return 2;
    if (n % 3 == 0) return 3;
    ull c = uniform_int_distribution<ull>(1, n - 1)(rng);
    ull x = uniform_int_distribution<ull>(0, n - 1)(rng), y = x, d = 1;
    auto f = [&](ull x) { return (modMul(x, x, n) + c) % n; };
    while (d == 1) {
        x = f(x);
        y = f(f(y));
        d = gcd(x > y ? x - y : y - x, n);
    }
    return d == n ? rho(n) : d;
} // a9272a

void factor(ull n, vector<ull> &fs) {
    if (n == 1) return;
    if (isPrime(n)) { fs.pb(n); return; }
    ull d = rho(n);
    factor(d, fs);
    factor(n / d, fs);
} // 6d58e9
```

Estruturas de Dados

Fenwick Tree

Fenwick para soma em prefixo/range. Interface: construa `Fenwick ft(n)`; use `add(i, val)` para somar no índice `i`, `sum(r)` para obter a soma em $[0, r)$ e `sum(l, r)` para obter a soma em $[l, r)$. Complexidade: $O(\log n)$ por operação.

281281 - dsa/fenwick-tree.cpp - 14 lines

```
struct Fenwick {
    vector<ll> t;
    int n;
    Fenwick(int N) : t(N), n(N) {}
    void add(int i, ll val) {
        for (i++; i <= n; i += i & -i) t[i - 1] += val;
    } // 638501
    ll sum(int r) {
        ll s = 0;
        for (; r; r -= r & -r) s += t[r - 1];
        return s;
    } // 025015
    ll sum(int l, int r) { return sum(r) - sum(l); }
};
```

Segtree

Segtree para qualquer monoide. Interface: defina o tipo `S`, a função `op` e o neutro `e`; construa `Segtree<S, op, e> seg(n)`, faça updates pontuais com `set(i, val)` e consultas em $[l, r)$ com `query(l, r)`. Complexidade: $O(\log n)$ por operação.

b76e87 - dsa/segtree.cpp - 18 lines

```
template<class S, S (*op)(S, S), S (*e)()>
struct Segtree {
    vector<S> t;
    int n;
    Segtree(int N): t(2 * N, e()), n(N) {}
    void set(int i, S value) {
        t[i += n] = value;
        for (i >>= 1; i; i >>= 1) t[i] = op(t[i << 1], t[i << 1 | 1]);
    } // 630e57
    S query(int l, int r) {
        S al = e(), ar = e();
        for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
            if (l & 1) al = op(al, t[l++]);
            if (r & 1) ar = op(t[--r], ar);
        }
        return op(al, ar);
    } // 9ee903
};
```

Lazy Segtree

Segtree com lazy propagation. Interface: além de `S`, `op` e `e`, defina o tipo de update `U`, a aplicação `mapping`, a composição `compose` e a identidade `id`; construa

`LazySegtree<...> seg(n)`, use `set(i, val)`, `apply(l, r, upd)` e `sum(l, r)` em $[l, r)$. Complexidade: $O(\log n)$ por operação.

21bbc4 - dsa/lazy-segtree.cpp - 50 lines

```
template<class S, S (*op)(S, S), S (*e)(), class U, S (*mapping)(U, S),
         U (*compose)(U, U), U (*id)()>
struct LazySegtree {
    vector<S> t;
    vector<U> d;
    int n;
    LazySegtree(int N): t(4 * N, e()), d(4 * N, id()), n(N) {}
    void applyNode(int i, U u) {
        t[i] = mapping(u, t[i]);
        d[i] = compose(u, d[i]);
    } // 3e5d0d
    void push(int i) {
        applyNode(i << 1, d[i]);
        applyNode(i << 1 | 1, d[i]);
        d[i] = id();
    } // 7836a8
    void pull(int i) { t[i] = op(t[i << 1], t[i << 1 | 1]); }
    void set(int j, S val) { set(j, val, 0, n, 1); }
    void set(int j, S val, int tl, int tr, int i) {
        if (tl + 1 == tr) {
            t[i] = val;
            d[i] = id();
            return;
        }
        push(i);
        int tm = (tl + tr) / 2;
        if (j < tm) set(j, val, tl, tm, i << 1);
        else set(j, val, tm, tr, i << 1 | 1);
        pull(i);
    } // f9c2fd
    void apply(int l, int r, U u) { apply(l, r, u, 0, n, 1); }
    void apply(int l, int r, U u, int tl, int tr, int i) {
        if (r <= tl || tr <= l) return;
        if (l <= tl && tr <= r) return applyNode(i, u);
        push(i);
        int tm = (tl + tr) / 2;
        apply(l, r, u, tl, tm, i << 1);
        apply(l, r, u, tm, tr, i << 1 | 1);
        pull(i);
    } // 3f0fb7
    S sum(int l, int r) { return sum(l, r, 0, n, 1); }
    S sum(int l, int r, int tl, int tr, int i) {
        if (r <= tl || tr <= l) return e();
        if (l <= tl && tr <= r) return t[i];
        push(i);
        int tm = (tl + tr) / 2;
        return op(sum(l, r, tl, tm, i << 1),
                  sum(l, r, tm, tr, i << 1 | 1));
    } // b976f6
};
```

Sparse Table

Sparse table para operações idempotentes, como `min`, `max` e `gcd`. Interface: defina `op`, faça `SparseTable<S, op> st`, chame `build(v)` e consulte intervalos $[l, r)$ com `query(l, r)`. Construção: $O(n \log n)$. Consulta: $O(1)$.

2e4ed0 - dsa/sparse-table.cpp - 18 lines

```
template <class S, S (*op)(S, S)> struct SparseTable {
    vector<vector<S>> t;
    vi lg;
    void build(vector<S> &v) {
        int n = ssize(v);
        lg.assign(n + 1, 0);
        rep (i, 2, n + 1) lg[i] = lg[i >> 1] + 1;
        t.assign(lg[n] + 1, vector<S>(n));
        t[0] = v;
        rep (k, 1, lg[n] + 1)
            for (int i = 0; i + (1 << k) <= n; i++)
                t[k][i] = op(t[k - 1][i], t[k - 1][i + (1 << (k - 1))]);
    } // 9c11fe
    S query(int l, int r) {
        int k = lg[r - l];
        return op(t[k][l], t[k][r - (1 << k)]);
    } // d2d23e
};
```

Disjoint Set Union

DSU com compressão de caminho e union by size. Interface: construa `DSU d(n)`, use `root(x)` para obter o representante e `merge(x, y)` para unir dois conjuntos; o retorno de `merge` indica se a união realmente aconteceu. Complexidade amortizada: $O(\alpha(n))$ por operação.

49a155 - dsa/dsu.cpp - 17 lines

```
struct DSU {
    vi par, cnt;
    DSU(int n): par(n), cnt(n, 1) {
        iota(all(par), 0);
    }
    int root(int x) {
        if (par[x] == x) return x;
        return par[x] = root(par[x]);
    } // 809557
    bool merge(int x, int y) {
        x = root(x), y = root(y);
        if (x == y) return false;
        if (cnt[x] < cnt[y]) swap(x, y);
        cnt[x] += cnt[y], par[y] = x;
        return true;
    } // 3bef8c
};
```

Convolução

Implementações de convolução rápida. Interface: passe os dois vetores/coefs para a rotina correspondente e receba o vetor resultante, onde $c_k = \sum_{i+j=k} a_i b_j$. Use FFT para reais/inteiros com cuidado numérico e NTT quando houver módulo adequado.

FFT

c8b36c - dsa/fft.cpp - 38 lines

```
using cd = complex<ld>;
const ld PI = acosl(-1.0L);

void fft(vector<cd> &a) {
    int n = sz(a), lg = 31 - __builtin_clz(n);
    static vector<cd> rt(2, 1);
    for (static int k = 2; k < n; k <<= 1) {
        rt.resize(n);
        cd x = polar((ld)1, PI / k);
        rep(i, k, 2 * k) rt[i] = i & 1 ? rt[i / 2] * x : rt[i / 2];
    }
    vi rev(n);
    rep(i, 0, n) rev[i] = (rev[i / 2] | ((i & 1) << lg)) / 2;
    rep(i, 0, n) if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k <<= 1)
        for (int i = 0; i < n; i += k << 1)
            rep(j, 0, k) {
                cd z = rt[j + k] * a[i + j + k];
                a[i + j + k] = a[i + j] - z;
                a[i + j] += z;
            }
} // c30ab4

vector<ld> conv(vector<ld> &a, vector<ld> &b) {
    if (a.empty() || b.empty()) return {};
    int s = sz(a) + sz(b) - 1, n = 1;
    while (n < s) n <<= 1;
    vector<cd> in(n), out(n);
    rep(i, 0, sz(a)) in[i].real(a[i]);
    rep(i, 0, sz(b)) in[i].imag(b[i]);
    fft(in);
    for (cd &x : in) x *= x;
    rep(i, 0, n) out[i] = in[-i & (n - 1)] - conj(in[i]);
    fft(out);
    vector<ld> res(s);
    rep(i, 0, s) res[i] = imag(out[i]) / (4 * n);
    return res;
} // 6d98ab
```

NTT

0394d1 - dsa/ntt.cpp - 40 lines

```
// const int MOD = 998244353, ROOT = 62;
void ntt(vector<ll> &a) {
    int n = sz(a);
    static array<ll, 1 << 23> r = {1, 1};
    for (static int t = 2, s = 2; t < n; t <<= 1, s++) {
        array z = {1ll, powm(ROOT, MOD >> s)};
        for (int i = t; i < t << 1; i++) r[i] = r[i >> 1] * z[i & 1] % MOD;
    }
    static array<ll, 1 << 23> b, c;
    copy(all(a), b.begin());
    for (int m = n; m >>= 1;) {
        for (int l = 0; l < n; l += m << 1)
            rep (i, 0, m)
                c[l + i] = b[l + 2 * i], c[l + m + i] = b[l + 2 * i + 1];
```

```
        copy(c.begin(), c.begin() + n, b.begin());
    }
    for (int m = 1; m < n; m <<= 1) {
        for (int l = 0; l < n; l += m << 1)
            rep (i, 0, m) {
                ll z = r[m + i] * b[l + m + i] % MOD;
                c[l + i] = b[l + i] + z;
                c[l + m + i] = b[l + i] + (MOD - z);
            }
        rep (i, 0, n) b[i] = c[i] < MOD ? c[i] : c[i] - MOD;
    }
    rep(i, 0, n) a[i] = b[i];
} // 623ed9

vector<ll> conv(vector<ll> &a, vector<ll> &b) {
    if (a.empty() || b.empty()) return {};
    int s = sz(a) + sz(b) - 1, n = 1;
    while (n < s) n <<= 1;
    int inv = powm(n, MOD - 2);
    vector<ll> L(a), R(b), out(n);
    L.resize(n), R.resize(n);
    ntt(L), ntt(R);
    rep(i, 0, n) out[-i & (n - 1)] = L[i] * R[i] % MOD * inv % MOD;
    ntt(out);
    return {out.begin(), out.begin() + s};
} // 3fa97a
```

Convolução de XOR

FWHT para convolução por XOR. Interface: chame `xorConv(a, b)`; a função redimensiona os vetores para o menor tamanho potência de 2 suficiente e retorna `c`, onde $c[k] = \sum_i a[i] \cdot b[i \oplus k]$. Use quando os índices representam máscaras. O snippet está em inteiros; as linhas comentadas mostram como adaptar para módulo. Complexidade: $O(n \log n)$.

fd3fd7 - dsa/xor-convolution.cpp - 20 lines

```
void fwht(vector<ll> &a) {
    for (int k = 1; k < sz(a); k <<= 1)
        for (int i = 0; i < sz(a); i += 2 * k)
            rep (j, i, i + k) {
                ll x = a[j], y = a[j + k];
                a[j] = x + y; // mod: a[j] = (x + y) % MOD;
                a[j + k] = x - y; // mod: a[j + k] = (x - y + MOD) % MOD;
            }
} // ae23e1

vector<ll> xorConv(vector<ll> a, vector<ll> b) {
    int n = 1;
    while (n < max(sz(a), sz(b))) n <<= 1;
    a.resize(n), b.resize(n);
    fwht(a), fwht(b);
    rep (i, 0, n) a[i] *= b[i]; // mod: a[i] = a[i] * b[i] % MOD;
    fwht(a);
    rep (i, 0, n) a[i] /= n; // mod: a[i] = a[i] * invN % MOD;
    return a;
} // 0bc2c7
```

Max Add Segtree

Interface: construa `MaxAddSegtree seg(n)`. Use `update(l, r, v)` para somar `v` em $[l, r)$ e `query(l, r)` para obter o máximo no intervalo. Complexidade: $O(\log n)$ por operação.

c008ec - dsa/max-add-segtree.cpp - 41 lines

```
struct MaxAddSegtree {
    int n, h;
    vector<i64> t, d;
    MaxAddSegtree(int n) : n(n), h(32 - __builtin_clz(n)), t(2 * n), d(n) {}
    void apply(int p, i64 v) {
        t[p] += v;
        if (p < n) d[p] += v;
    } // 94936b
    void build(int p) {
        while (p > 1) p >>= 1, t[p] = max(t[p<<1], t[p<<1|1]) + d[p];
    } // c20980
    void push(int p) {
        for (int s = h; s > 0; --s) {
            int i = p >> s;
            if (d[i]) {
                apply(i<<1, d[i]);
                apply(i<<1|1, d[i]);
                d[i] = 0;
            }
        }
    } // 452305
    void update(int l, int r, i64 v) {
```

```
        l += n, r += n;
        int l0 = l, r0 = r;
        for (; l < r; l >>= 1, r >>= 1) {
            if (l & 1) apply(l++, v);
            if (r & 1) apply(--r, v);
        }
        build(l0); build(r0 - 1);
    } // 8a7429
    i64 query(int l, int r) {
        l += n, r += n;
        push(l); push(r - 1);
        i64 res = -1e18;
        for (; l < r; l >>= 1, r >>= 1) {
            if (l & 1) res = max(res, t[l++]);
            if (r & 1) res = max(res, t[--r]);
        }
        return res;
    } // f655b5
};
```

Zeta e Mobius

Zeta e Möbius sobre subconjuntos. Interface: use o vetor indexado por máscaras como entrada; a transformada zeta acumula contribuições de submáscaras e a de Möbius desfaz essa operação. Útil em SOS DP e contagens por subconjunto. Complexidade: $O(B2^B)$ para máscaras de B bits.

eff0df - dsa/zeta.cpp - 28 lines

```
vi subsetZeta(vi a) {
    int n = sz(a);
    for (int j = 1; j < n; j <<= 1)
        rep (i, 1, n)
            if (i & j) a[i] += a[i ^ j];
    return a;
} // b39c8d
vi subsetMobius(vi a) {
    int n = sz(a);
    for (int j = 1; j < n; j <<= 1)
        rep(i, 1, n)
            if (i & j) a[i] -= a[i ^ j];
    return a;
} // 5ffc86
vi supersetZeta(vi a) {
    int n = sz(a);
    for (int j = 1; j < n; j <<= 1)
        rep(i, 1, n)
            if (i & j) a[i ^ j] += a[i];
    return a;
} // c5d556
vi supersetMobius(vi a) {
    int n = sz(a);
    for (int j = 1; j < n; j <<= 1)
        rep(i, 1, n)
            if (i & j) a[i ^ j] -= a[i];
    return a;
} // 128136
```

Árvore de Li Chao

Li Chao para mínimo em domínio discreto $[0, N)$. Interface: construa `LiChao lc(N)`, insira retas `{a, b}` com `insert(line)` e consulte em `x` com `query(x)`; o retorno é a melhor reta, então use `line(x)` para obter o valor. Complexidade: $O(\log N)$ por inserção/consulta.

9e88be - dsa/li-chao.cpp - 29 lines

```
struct Line {
    ll a, b;
    ll operator()(int x) { return a * x + b; }
};
struct LiChao {
    vector<Line> t;
    int n;
    LiChao(int N): t(4 * N, {0, LLONG_MAX}), n(N) {} // Replace with LLONG_MIN in
    ↪  case of finding max
    void insert(Line line) { insert(line, 0, n, 1); }
    void insert(Line line, int tl, int tr, int i) {
        int tm = tl + (tr - tl) / 2;
        if (line(tm) < t[i](tm)) swap(line, t[i]); // Replace with > in case of
        ↪  finding max
        if (tl + 1 == tr) return;
        if (t[i](tl) < line(tl)) insert(line, tm, tr, i << 1 | 1); // Replace with >
        ↪  in case of finding max
        else insert(line, tl, tm, i << 1);
    } // f784a4
    Line query(int x) {
        Line ans = {0, LLONG_MAX}; // Replace with LLONG_MIN in case of finding max
```

```
        query(x, ans, 0, n, 1);
        return ans;
    } // 2aca6b
    void query(int x, Line &line, int tl, int tr, int i) {
        if (t[i](x) < line(x)) line = t[i]; // Replace with > in case of finding max
        int tm = tl + (tr - tl) / 2;
        if (tl + 1 == tr) return;
        if (x < tm) query(x, line, tl, tm, i << 1);
        else query(x, line, tm, tr, i << 1 | 1);
    } // 973d96
};
```

Árvore de Li Chao++

Versão segmentada da Li Chao. Interface: construa `LiChao lc(N)`, use `insert(line, L, R)` para ativar a reta apenas em $[L, R)$ e `query(x)` para obter a melhor reta em `x`. Complexidade: $O(\log^2 N)$ por inserção e $O(\log N)$ por consulta.

e28ce0 - dsa/li-chao-extended.cpp - 36 lines

```
struct Line {
    ll a, b;
    ll operator()(int x) { return a * x + b; }
};
struct LiChao {
    vector<Line> t;
    int n;
    LiChao(int N): t(4 * N, {0, LLONG_MAX}), n(N) {} // Replace with LLONG_MIN in
    ↪  case of finding max
    void insert(Line line, int L, int R) { insert(line, L, R, 0, n, 1); }
    void insert(Line line, int L, int R, int tl, int tr, int i) {
        int tm = tl + (tr - tl) / 2;
        if (L >= tr || R <= tl) return;
        if (L <= tl && tr <= R) {
            if (line(tm) < t[i](tm)) swap(line, t[i]); // Replace with > in case of
            ↪  finding max
            if (tl + 1 == tr) return;
            if (t[i](tl) < line(tl)) insert(line, L, R, tm, tr, i << 1 | 1); //
            ↪  Replace with > in case of finding max
            else insert(line, L, R, tl, tm, i << 1);
        } else {
            if (tl + 1 == tr) return;
            insert(line, L, R, tm, tr, i << 1 | 1);
            insert(line, L, R, tl, tm, i << 1);
        }
    } // 3dcc38
    Line query(int x) {
        Line ans = {0, LLONG_MAX}; // Replace with LLONG_MIN in case of finding max
        query(x, ans, 0, n, 1);
        return ans;
    } // 2aca6b
    void query(int x, Line &line, int tl, int tr, int i) {
        if (t[i](x) < line(x)) line = t[i]; // Replace with > in case of finding max
        int tm = tl + (tr - tl) / 2;
        if (tl + 1 == tr) return;
        if (x < tm) query(x, line, tl, tm, i << 1);
        else query(x, line, tm, tr, i << 1 | 1);
    } // 973d96
};
```

Árvore Cartesiana

Constrói a árvore cartesiana dos índices $0, \ldots, n-1$. Interface: passe `n` e um comparador `le(i, j)` dizendo quando `i` deve ficar acima de `j`; o retorno é o vetor `par`, onde `par[v]` é o pai de `v`. Complexidade: $O(n)$.

caeaba - dsa/cartesian-tree.cpp - 20 lines

```
vi cartTree(int n, auto le) {
    int last;
    vi st, par(n);
    rep (i, 0, n) {
        par[i] = i, last = -1;
        while (sz(st) && le(par[i], st.back())) {
            if (last != -1) par[last] = st.back();
            last = st.back();
            st.pop_back();
        }
        if (last != -1) par[last] = i;
        st.pb(i);
    }
    while (sz(st) > 1) {
        last = st.back();
        st.pop_back();
        par[last] = st.back();
    }
    return par;
} // caeaba
```

Árvore de Busca Binária Aleatória

Base de RBST genérica. Interface: a estrutura mantém a raiz em `rt` e expõe primitivas como `merge`, `split`, `splitAt`, `front` e `back`; o `main` do arquivo mostra como montar um conjunto ordenado a partir delas. Complexidade esperada: $O(\log n)$ por operação.

6a6cd3 - dsa/rbst.cpp - 65 lines

```
static mt19937 rng;

template<class T>
struct RBST {
    struct Node {
        T v;
        int cnt = 1;
        Node *ch[2] = {0, 0};
        ~Node() {
            for (int i: {0, 1}) if (ch[i]) delete ch[i];
        }
    };
    static Node * update(Node *n) {
        if (!n) return 0;
        n->cnt = 1;
        for (int i: {0, 1}) n->cnt += n->ch[i] ? n->ch[i]->cnt : 0;
        return n;
    } // 0fee8d
    static Node *merge(Node *l, Node *r) {
        if (!l || !r) return l ? l : r;
        if (rng() % (l->cnt + r->cnt) < l->cnt) {
            l->ch[1] = merge(l->ch[1], r);
            return update(l);
        } else {
            r->ch[0] = merge(l, r->ch[0]);
            return update(r);
        }
    } // f0d9c8
    static void split(Node *rt, auto cond, Node *&l, Node *&r) {
        if (!rt) return;
        if (cond(rt->v)) {
            r = rt, rt = r->ch[0], r->ch[0] = 0;
            split(rt, cond, l, r->ch[0]);
            update(r);
        } else {
            l = rt, rt = l->ch[1], l->ch[1] = 0;
            split(rt, cond, l->ch[1], r);
            update(l);
        }
    } // 44f1a1
    static void splitAt(Node *rt, int i, Node *&l, Node *&r) {
        if (!rt) return;
        int k = rt->ch[0] ? rt->ch[0]->cnt : 0;
        if (i <= k) {
            r = rt, rt = r->ch[0], r->ch[0] = 0;
            splitAt(rt, i, l, r->ch[0]);
            update(r);
            return;
        }
        i -= k + 1;
        l = rt, rt = l->ch[1], l->ch[1] = 0;
        splitAt(rt, i, l->ch[1], r);
        update(l);
    } // a19d8b
    static Node *front(Node * rt) {
        if (!rt || !rt->ch[0]) return rt;
        return front(rt->ch[0]);
    } // b1a234
    static Node *back(Node *rt) {
        if (!rt || !rt->ch[1]) return rt;
        return back(rt->ch[1]);
    } // 441301
    Node *rt = 0;
    RBST() : rt(0) {}
};
```

Treap implícita

Treap implícita para representar uma sequência. Interface: use `split(t, k, a, b)` e `merge(a, b)` para quebrar/juntar por posição; há helpers prontos como `ins(rt, p, v)`, `era(rt, p)`, `kth(t, p)` e `sep(rt, l, r, a, b, c)`. Também suporta reversão lazy e soma em intervalo. Complexidade esperada: $O(\log n)$ por operação.

ad716f - dsa/treap.cpp - 69 lines

```
struct Node;
using TN = Node*;

struct Node {
    int y = rng(), cnt, v;
    ll sum;
```

```
    bool rev = 0;
    TN l = 0, r = 0;
    Node(int v) : cnt(1), v(v), sum(v) {}
};

int cnt(TN t) { return t ? t->cnt : 0; }
ll sum(TN t) { return t ? t->sum : 0; }

void apply_rev(TN t) {
    if (!t) return;
    t->rev ^= 1;
    swap(t->l, t->r);
} // 907c51
void push(TN t) {
    if (!t || !t->rev) return;
    apply_rev(t->l);
    apply_rev(t->r);
    t->rev = 0;
} // b46a2a
void pull(TN t) {
    t->cnt = 1 + cnt(t->l) + cnt(t->r);
    t->sum = t->v + sum(t->l) + sum(t->r);
} // 6ee5eb

void split(TN t, int k, TN& a, TN& b) {
    if (!t) return a = b = 0, void();
    push(t);
    if (cnt(t->l) >= k) split(t->l, k, a, t->l), b = t;
    else split(t->r, k - cnt(t->l) - 1, t->r, b), a = t;
    pull(t);
} // b34b6d
TN merge(TN a, TN b) {
    if (!a || !b) return a ? a : b;
    if (a->y > b->y) return push(a), a->r = merge(a->r, b), pull(a), a;
    return push(b), b->l = merge(a, b->l), pull(b), b;
} // 529963

void ins(TN& t, int p, int v) {
    TN a, b;
    split(t, p, a, b);
    t = merge(merge(a, new Node(v)), b);
} // c2b578
void era(TN& t, int p) {
    TN a, b, c;
    split(t, p, a, b);
    split(b, 1, b, c);
    delete b;
    t = merge(a, c);
} // cf32a5
TN kth(TN t, int p) {
    while (t) {
        push(t);
        if (cnt(t->l) == p) return t;
        if (cnt(t->l) > p) t = t->l;
        else p -= cnt(t->l) + 1, t = t->r;
    }
    return 0;
} // 4ba488

void sep(TN t, int l, int r, TN& a, TN& b, TN& c) {
    split(t, r, b, c);
    split(b, l, a, b);
} // 4fa0ee
```

Treap de conjunto ordenado

Treap de BST ordenada. Interface: mantenha a raiz em `TS* t = 0` e use `insert`, `erase`, `find`, `kth` (1-indexado), `cntLE`, `pred` e `succ`. Complexidade esperada: $O(\log n)$ por operação.

6ed571 - dsa/treap-set.cpp - 81 lines

```
struct TS {
    i64 v;
    i32 pr, sz;
    TS *l, *r;
    TS(i64 V) : v(V), pr((rand()<<16)^rand()), sz(1), l(0), r(0) {}
};

int sz(TS* t) { return t ? t->sz : 0; }
void pull(TS* t) { if (t) t->sz = 1 + sz(t->l) + sz(t->r); }

void split(TS* t, int key, TS*& l, TS*& r) {
    if (!t) l = r = 0;
    else if (t->v <= key) split(t->r, key, t->r, r), l = t;
    else split(t->l, key, l, t->l), r = t;
    pull(t);
} // 0240c2

TS* merge(TS* l, TS* r) {
```

```
    if (!l || !r) return l ? l : r;
    if (l->pr > r->pr) {
        l->r = merge(l->r, r);
        pull(l);
        return l;
    } else {
        r->l = merge(l, r->l);
        pull(r);
        return r;
    }
} // 244be4

bool find(TS* t, int v) {
    while (t) {
        if (t->v == v) return 1;
        t = v < t->v ? t->l : t->r;
    }
    return 0;
} // 3b0421

void insert(TS*& t, TS* it) {
    if (!t) t = it;
    else if (it->pr > t->pr) split(t, it->v, it->l, it->r), t = it;
    else insert(it->v < t->v ? t->l : t->r, it);
    pull(t);
} // 75b7ab

void erase(TS*& t, int v) {
    if (!t) return;
    if (t->v == v) t = merge(t->l, t->r);
    else erase(v < t->v ? t->l : t->r, v);
    pull(t);
} // 1f8476

int kth(TS* t, int k) { // 1-indexed, assumes 1 <= k <= sz(t)
    if (sz(t->l) + 1 == k) return t->v;
    if (k <= sz(t->l)) return kth(t->l, k);
    return kth(t->r, k - sz(t->l) - 1);
} // 41972c

int cntLE(TS* t, i64 v) {
    if (!t) return 0;
    if (t->v <= v) return sz(t->l) + 1 + cntLE(t->r, v);
    return cntLE(t->l, v);
} // 8e7469

TS* pred(TS* t, i64 v) { // max key <= x
    TS* ans = 0;
    while (t) {
        if (t->v <= v) ans = t, t = t->r;
        else t = t->l;
    }
    return ans;
} // 9dda3d

TS* succ(TS* t, i64 v) { // min key >= x
    TS* ans = 0;
    while (t) {
        if (t->v >= v) ans = t, t = t->l;
        else t = t->r;
    }
    return ans;
} // dcf6b7
```

Grafos

Dijkstra

Menores caminhos com pesos não negativos. Interface: passe a adjacência `g`, onde cada aresta é `{to, w}`, e a origem `s`; o retorno é o vetor `dist`. Vértices inalcançáveis ficam com um valor grande. O snippet retorna só as distâncias; se precisar reconstruir caminho, mantenha o pai fora da rotina. Complexidade: $O((V + E) \log V)$.

4bb2a0 - graph/dijkstra.cpp - 18 lines

```
using pil = pair<int, ll>;
vector<ll> dijkstra(vector<vector<pil>> &g, int s) {
    const ll INF = 1LL << 62;
    vector<ll> dist(sz(g), INF);
    priority_queue<pll> q;
    dist[s] = 0;
    q.push({0, s});
    while (sz(q)) {
        auto [nd, v] = q.top(); q.pop();
        ll d = -nd;
        if (d != dist[v]) continue;
        for (auto [to, w] : g[v]) if (d + w < dist[to]) {
            dist[to] = d + w;
            q.push({-dist[to], to});
```

```
        }
    }
    return dist;
} // 781494
```

0-1 BFS

Menores caminhos quando cada aresta tem peso `0` ou `1`. Interface: passe a adjacência `g` nesse formato e a origem `s`; o retorno é `dist`, com `dist[v]` igual ao menor custo até `v`. Vértices inalcançáveis ficam com um valor grande. Complexidade: $O(V + E)$.

72f0a3 - graph/bfs01.cpp - 16 lines

```
vi bfs01(vector<vector<pii>> &g, int s) {
    const int INF = 1e9;
    vi dist(sz(g), INF);
    deque<int> q;
    dist[s] = 0;
    q.push_back(s);
    while (sz(q)) {
        int v = q.front(); q.pop_front();
        for (auto [to, w] : g[v]) if (dist[v] + w < dist[to]) {
            dist[to] = dist[v] + w;
            if (w) q.push_back(to);
            else q.push_front(to);
        }
    }
    return dist;
} // 72f0a3
```

Caminho Euleriano

Hierholzer para caminho/circuito euleriano. Interface: passe uma lista de adjacência no formato `{vizinho, idDaAresta}`; em grafos não direcionados, use o mesmo id nas duas direções da aresta. O retorno é a ordem dos vértices/arestas do caminho encontrado. Complexidade: $O(V + E)$.

483eba - graph/eulerian-walk.cpp - 23 lines

```
pair<vi, vi> eulerWalk(vector<vector<pii>>& g, int m, int s = 0) {
    int n = sz(g);
    vi path, walk, d(n), vis(m), it(n);
    vector<pii> st = {{s, -1}};
    d[s]++; // remove to force cycles
    while (sz(st)) {
        auto [v, e] = st.back();
        if (it[v] == sz(g[v])) {
            path.pb(v), walk.pb(e);
            st.pop_back();
        } else {
            auto [u, id] = g[v][it[v]++];
            if (!vis[id]) {
                d[v]--, d[u]++;
                vis[id] = 1;
                st.eb(u, id);
            }
        }
    }
    for (int x : d) if (x < 0) return {{}, {}};
    if (sz(walk) != m + 1) return {{}, {}};
    return { vi(path.rbegin(), path.rend()), vi(walk.rbegin() + 1, walk.rend()) };
} // 483eba
```

Árvore de pontes

DFS de lowlinks para grafo não direcionado. Interface: passe a adjacência `g`; `bridges(g)` retorna `comp`, onde `comp[v]` é a componente de `v` na árvore de pontes. Uma aresta `(u, v)` é ponte sse `comp[u] != comp[v]`. Complexidade: $O(V + E)$.

c3d125 - graph/bridges.cpp - 31 lines

```
vi bridges(vector<vi> &g) {
    int n = sz(g), ti = 0, cc = 0;
    vi tin(n), low(n), st, comp;
    comp.assign(n, -1);
    auto make = [&](int v) {
        while (1) {
            int x = st.back(); st.pop_back();
            comp[x] = cc;
            if (x == v) break;
        }
        cc++;
    };
    auto dfs = [&](auto self, int v, int p) -> void {
        tin[v] = low[v] = ++ti;
        st.pb(v);
        int usedPar = 0;
```

```
        for (int to : g[v]) {
            if (to == p && !usedPar) { usedPar = 1; continue; }
            if (!tin[to]) {
                self(self, to, v);
                low[v] = min(low[v], low[to]);
                if (low[to] > tin[v]) make(to);
            } else low[v] = min(low[v], tin[to]);
        }
    };
    rep(i,0,n) if (!tin[i]) {
        dfs(dfs, i, -1);
        make(i);
    }
    return comp;
} // c3d125
```

Árvore de bloco e corte

Interface: passe um grafo não direcionado `g`; `bct(g)` retorna a árvore de bloco e corte `t`. Os nós `0..n-1` continuam sendo os vértices originais, e cada nó novo `i >= n` representa uma componente biconexa. Assim, `t[v]` lista as componentes biconexas que contêm o vértice original `v`, e `t[id]` lista os vértices originais pertencentes ao bloco `id`. Em particular, `t[v].size() > 1` sse `v` é ponto de articulação. Componentes isoladas de um único vértice também geram um nó de bloco. Complexidade: $O(V + E)$.

7605c4 - graph/block-cut-tree.cpp - 35 lines

```
vector<vi> bct(vector<vi> &g) {
    int n = sz(g), ti = 0;
    vector<vi> t(n);
    vi num(n), low(n), seen(n);
    vector<pii> st;
    auto add = [&](int a, int b) {
        int id = sz(t), it = id + 1;
        t.pb({});
        while (1) {
            auto [x, y] = st.back(); st.pop_back();
            if (seen[x] != it) seen[x] = it, t[x].pb(id), t[id].pb(x);
            if (seen[y] != it) seen[y] = it, t[y].pb(id), t[id].pb(y);
            if (x == a && y == b) break;
        }
    };
    auto dfs = [&](auto self, int v, int p) -> void {
        num[v] = low[v] = ++ti;
        int ch = 0;
        for (int to : g[v]) if (to != p) {
            if (!num[to]) {
                st.eb(v, to);
                ch++;
                self(self, to, v);
                low[v] = min(low[v], low[to]);
                if (low[to] >= num[v]) add(v, to);
            } else if (num[to] < num[v]) {
                st.eb(v, to);
                low[v] = min(low[v], num[to]);
            }
        }
        if (p == -1 && !ch) t.pb({v}), t[v].pb(sz(t) - 1);
    };
    rep(i, 0, n) if (!num[i]) dfs(dfs, i, -1);
    return t;
} // 7605c4
```

Dominator Tree

Lengauer–Tarjan para grafo dirigido. Interface: passe a adjacência `g` e a raiz `s`; o retorno é `idom`, onde `idom[v]` é o dominador imediato de `v` no subgrafo alcançável a partir de `s`. O próprio `s` sai com `idom[s] = s`, e vértices inalcançáveis ficam com `-1`. Complexidade: quase linear, $O((V + E)\alpha(V))$ na prática.

72f9c4 - graph/dominator-tree.cpp - 34 lines

```
vi domTree(vector<vi> &g, int s) {
    int n = sz(g), t = 0;
    vector<vi> rg(n + 1), bucket(n + 1);
    vi arr(n), rev(n + 1), par(n + 1), sdom(n + 1), dom(n + 1), dsu(n + 1), mn(n +
    ↪ 1);
    auto dfs = [&](auto &&self, int u) -> void {
        arr[u] = ++t; rev[t] = u;
        sdom[t] = dom[t] = dsu[t] = mn[t] = t;
        for (int v : g[u]) {
            if (!arr[v]) self(self, v), par[arr[v]] = arr[u];
            rg[arr[v]].pb(arr[u]);
        }
    };
    auto find = [&](auto &&self, int u, int x = 0) -> int {
        if (u == dsu[u]) return x ? -1 : u;
        int v = self(self, dsu[u], x + 1);
```

```
        if (v < 0) return u;
        if (sdom[mn[dsu[u]]] < sdom[mn[u]]) mn[u] = mn[dsu[u]];
        return dsu[u] = v, x ? v : mn[u];
    };
    dfs(dfs, s);
    for (int i = t; i; i--) {
        for (int j : rg[i]) sdom[i] = min(sdom[i], sdom[find(find, j)]);
        if (i > 1) bucket[sdom[i]].pb(i);
        for (int j : bucket[i]) {
            int y = find(find, j);
            dom[j] = sdom[y] == sdom[j] ? sdom[j] : y;
        }
        if (i > 1) dsu[i] = par[i];
    }
    rep(i, 2, t + 1) if (dom[i] != sdom[i]) dom[i] = dom[dom[i]];
    vi idom(n, -1);
    rep(i, 1, t + 1) idom[rev[i]] = rev[dom[i]];
    return idom;
} // 72f9c4
```

Árvore geradora mínima direcionada

Chu–Liu/Edmonds para MST direcionada. Interface: monte o vetor `e` de arestas no formato `E{a, b, w}`, onde `a -> b` tem custo `w`, e chame `dmst(n, r, e)`. O retorno é o custo mínimo de uma arborescência que alcança todos os vértices a partir da raiz `r`; se algum vértice não puder ser alcançado a partir de `r`, a função retorna `-1`. O snippet retorna só o custo, não as arestas escolhidas. Complexidade: $O(VE)$.

be21b7 - graph/directed-mst.cpp - 37 lines

```
struct E {
    int a, b;
    ll w;
};

ll dmst(int n, int r, vector<E> e) {
    const ll INF = 1LL << 62;
    ll ans = 0;
    while (1) {
        vector<ll> in(n, INF);
        vi pre(n, -1), id(n, -1), vis(n, -1);
        for (auto [a, b, w] : e) if (a != b && w < in[b]) in[b] = w, pre[b] = a;
        in[r] = 0;
        rep (i, 0, n) if (in[i] == INF) return -1;
        int cnt = 0;
        rep (i, 0, n) {
            ans += in[i];
            int v = i;
            while (vis[v] != i && id[v] == -1 && v != r) vis[v] = i, v = pre[v];
            if (v != r && id[v] == -1) {
                id[v] = cnt;
                for (int u = pre[v]; u != v; u = pre[u]) id[u] = cnt;
                cnt++;
            }
        }
        if (!cnt) break;
        rep (i, 0, n) if (id[i] == -1) id[i] = cnt++;
        vector<E> ne;
        for (auto [a, b, w] : e) {
            int x = id[a], y = id[b];
            if (x != y) ne.pb({x, y, w - in[b]});
        }
        r = id[r], n = cnt;
        e.swap(ne);
    }
    return ans;
} // 17980e
```

Encontrando qualquer caminho euleriano

Em grafos direcionados:

- Se exatamente um vértice tem $d_{out} - d_{in} = 1$, então comece nesse vértice.
- Se todos os vértices tem $d_{out} = d_{in}$, pode começar em qualquer vértice com $d_{out} > 0$.
- Caso contrário, não existe caminho euleriano.

Em grafos não direcionados:

- Se exatamente dois vértices tem deg ímpar, então comece em qualquer um deles.
- Se todos os vértices tem deg par, pode começar em qualquer vértice com $deg > 0$.
- Caso contrário, não existe caminho euleriano.

Floresta de grafo funcional

Interface: construa `FuncForest ff(f)`, onde `f[i]` é o único vizinho de saída de `i` no grafo funcional. O snippet copia esse vetor para `ff.par`. Depois da construção, `ff.root[i]` guarda um representante do ciclo alcançado a partir de `i`; dois vértices estão na mesma componente funcional sse têm o mesmo `root`. Útil para agrupar

vértices por ciclo destino antes de montar lifting, DP reversa ou contagens por componente. Complexidade: $O(n)$.

c1f247 - graph/functional-forest.cpp - 20 lines

```
struct FuncForest {
    vi par, root;
    FuncForest(vi f) : par(f), root(sz(f), -1) {
        vi deg(sz(f)), ord;
        for (int v : f) deg[v]++;
        queue<int> q;
        rep (i, 0, sz(f)) if (!deg[i]) q.push(i);
        while (sz(q)) {
            int v = q.front(); q.pop();
            ord.pb(v);
            if (!--deg[par[v]]) q.push(par[v]);
        }
        rep (i, 0, sz(f)) if (deg[i]) {
            root[i] = i;
            for (int v = par[i]; v != i; v = par[v]) root[v] = i, deg[v] = 0;
            deg[i] = 0;
        }
        for (int i = sz(ord) - 1; i >= 0; --i) root[ord[i]] = root[par[ord[i]]];
    }
};
```

Componentes fortemente conexas e grafo de condensação

Kosaraju para SCC. Interface: passe a lista de adjacência dirigida `g`; `scc(g)` retorna `{cnt, comp}`, onde `cnt` é o número de componentes e `comp[v]` é a componente de `v`. Depois, use `condGraph(g, cnt, comp)` para construir o grafo de condensação sem arestas repetidas. Complexidade: $O(V + E)$.

2c057a - graph/scc.cpp - 31 lines

```
pair<int, vi> scc(vector<vi> &g) {
    int n = sz(g);
    vector<vi> rg(n);
    rep (v, 0, n) for (int u : g[v]) rg[u].pb(v);
    vi vis(n), ord, comp(n, -1);
    auto dfs1 = [&](auto self, int v) -> void {
        vis[v] = 1;
        for (int u : g[v]) if (!vis[u]) self(self, u);
        ord.pb(v);
    };
    auto dfs2 = [&](auto self, int v, int c) -> void {
        comp[v] = c;
        for (int u : rg[v]) if (comp[u] < 0) self(self, u, c);
    };
    rep (i, 0, n) if (!vis[i]) dfs1(dfs1, i);
    reverse(all(ord));
    int cnt = 0;
    for (int v : ord) if (comp[v] < 0) dfs2(dfs2, v, cnt++);
    return {cnt, comp};
} // a8c254

vector<vi> condGraph(vector<vi> &g, int cnt, vi &comp) {
    vector<vi> dag(cnt);
    rep (v, 0, sz(g)) for (int u : g[v])
        if (comp[v] != comp[u]) dag[comp[v]].pb(comp[u]);
    rep (i, 0, cnt) {
        sort(all(dag[i]));
        dag[i].erase(unique(all(dag[i])), end(dag[i]));
    }
    return dag;
} // a5cf62
```

2-SAT

Resolvedor de 2-SAT sobre variáveis 0-indexadas. Interface: crie `TwoSat ts(n)`, adicione cada cláusula $(x_a = va) \vee (x_b = vb)$ com `clause(a, va, b, vb)` e finalize com `solve()`. Se existir solução, o retorno é verdadeiro e a atribuição escolhida fica em `ts.val[i]`. Complexidade: $O(V + E)$ no grafo de implicação.

1412b8 - graph/2-sat.cpp - 22 lines

```
struct TwoSat {
    int n;
    vector<vi> g;
    vi val;
    TwoSat(int n) : n(n), g(2 * n) {}
    int lit(int x, bool v) { return 2 * x + v; }
    void imp(int a, int b) { g[a].pb(b); }
    void clause(int a, bool va, int b, bool vb) {
        int x = lit(a, va), y = lit(b, vb);
        imp(x ^ 1, y);
        imp(y ^ 1, x);
    } // 47d6a6
```

```
    bool solve() {
        auto [cnt, comp] = scc(g);
        val.assign(n, 0);
        rep (i, 0, n) {
            if (comp[2 * i] == comp[2 * i + 1]) return 0;
            val[i] = comp[2 * i] < comp[2 * i + 1];
        }
        return 1;
    } // a54ac0
};
```

Matching bipartido máximo

Hopcroft–Karp para matching bipartido máximo. Interface: passe a adjacência `g` da partição esquerda para a direita, além de vetores `l` e `r` inicializados com `-1`; a função `hopKarp(g, l, r)` devolve o tamanho do matching máximo e escreve os pares em `l[a]` e `r[b]`. Use `g.size()` como tamanho do lado esquerdo e `r.size()` como tamanho do lado direito. Complexidade: $O(E\sqrt{V})$.

2d4485 - graph/hopcroft-karp.cpp - 31 lines

```
int hopKarp(vector<vi> &g, vi &l, vi &r) {
    int n = sz(l), m = sz(r);
    vi d(n);
    auto bfs = [&]() {
        queue<int> q;
        rep(i, 0, n) d[i] = l[i] < 0 ? q.push(i), 0 : -1;
        bool f = 0;
        while (sz(q)) {
            int v = q.front(); q.pop();
            for (int u : g[v]) {
                int w = r[u];
                if (w < 0) f = 1;
                else if (d[w] < 0) d[w] = d[v] + 1, q.push(w);
            }
        }
        return f;
    };
    auto dfs = [&](auto &&self, int v) -> bool {
        for (int u : g[v]) {
            int w = r[u];
            if (w < 0 || (d[w] == d[v] + 1 && self(self, w))) {
                l[v] = u, r[u] = v;
                return 1;
            }
        }
        return d[v] = -1, 0;
    };
    int ans = 0;
    while (bfs()) rep(i, 0, n) ans += l[i] < 0 && dfs(dfs, i);
    return ans;
} // 2d4485
```

Max Flow

Dinic

Dinic para fluxo máximo. Interface:

- `addEdge(from, to, capacity)` adiciona uma aresta dirigida.
- `flow(s, t)` retorna o fluxo máximo de `s` para `t`.
- `flow(s, t, max)` faz o mesmo, mas limita o valor total a `max`.

Complexidade: $O(N^2M)$ no pior caso, e também limitada por $O(FM)$ quando o fluxo máximo é F.

f40f68 - graph/dinic.cpp - 55 lines

```
struct Dinic {
    struct Edge {
        ll s, t, f, c, r;
    };
    vector<Edge> e;
    vector<vi> adj;
    vi ne, lvl;
    Dinic(int n): adj(n), ne(n), lvl(n) {}
    void addEdge(int a, int b, ll c) {
        adj[a].pb(sz(e));
        e.eb(a, b, 0, c, sz(e) + 1);
        adj[b].pb(sz(e));
        e.eb(b, a, 0, 0, sz(e) - 2);
    } // 598446
    bool bfs(int s, int t) {
        queue<int> q;
        q.push(s);
        fill(all(lvl), -1);
        lvl[s] = 0;
        while (sz(q)) {
            int x = q.front();
            q.pop();
```

```
            for (int i: adj[x]) {
                if (e[i].f == e[i].c) continue;
                if (lvl[e[i].t] != -1) continue;
                lvl[e[i].t] = lvl[x] + 1;
                q.push(e[i].t);
            }
        }
        return lvl[t] == -1;
    } // 1625d2
    ll dfs(int x, int t, ll f) {
        if (f == 0) return 0;
        if (x == t) return f;
        for (int &te = ne[x]; te < sz(adj[x]); te++) {
            int i = adj[x][te], y = e[i].t;
            if (lvl[y] != lvl[x] + 1) continue;
            if (ll df = dfs(y, t, min(f, e[i].c - e[i].f))) {
                e[i].f += df;
                e[i ^ 1].f -= df;
                return df;
            }
        }
        return 0;
    } // d7b05b
    ll flow(int s, int t, ll mx = LLONG_MAX) {
        ll mf = 0;
        while (true) {
            if (bfs(s, t)) break;
            fill(all(ne), 0);
            while (ll f = dfs(s, t, mx - mf)) mf += f;
        }
        return mf;
    } // 53644f
};
```

Min Cost Flow

MCMF

Fluxo de custo mínimo com suporte a custos negativos. Interface:

- `addEdge(from, to, cap, cost)` adiciona uma aresta com capacidade e custo por unidade.
- `flow(s, t)` retorna `{max_flow, min_cost}`.

Complexidade: $O(VE \cdot flow)$ no pior caso.

e07252 - graph/mcf.cpp - 58 lines

```
struct MCMF {
    struct E { int to, rev; ll cap, cost; };
    int n;
    vector<vector<E>> g;
    MCMF(int n) : n(n), g(n) {}
    void addEdge(int u, int v, ll cap, ll cost) {
        g[u].eb(v, sz(g[v]), cap, cost);
        g[v].eb(u, sz(g[u]) - 1, 0, -cost);
    } // 82791b
    pll flow(int s, int t, ll k = 4e18) {
        const ll INF = 4e18;
        ll fl = 0, cost = 0;
        vector<ll> p(n, INF), d(n), f(n);
        vi pv(n), pe(n), inq(n);
        queue<int> q;
        p[s] = 0, q.push(s), inq[s] = 1;
        while (sz(q)) {
            int u = q.front(); q.pop();
            inq[u] = 0;
            rep(i,0,sz(g[u])) {
                auto &e = g[u][i];
                if (e.cap && p[e.to] > p[u] + e.cost) {
                    p[e.to] = p[u] + e.cost;
                    if (!inq[e.to]) q.push(e.to), inq[e.to] = 1;
                }
            }
        }
        rep(i,0,n) if (p[i] == INF) p[i] = 0;
        while (fl < k) {
            fill(all(d), INF);
            priority_queue<pll, vector<pll>, greater<pll>> pq;
            d[s] = 0, f[s] = k - fl, pq.emplace(0, s);
            while (sz(pq)) {
                auto [du, u] = pq.top(); pq.pop();
                if (du != d[u]) continue;
                rep(i,0,sz(g[u])) {
                    auto &e = g[u][i];
                    if (!e.cap) continue;
                    ll nd = du + e.cost + p[u] - p[e.to];
                    if (d[e.to] > nd) {
                        d[e.to] = nd, pv[e.to] = u, pe[e.to] = i;
                        f[e.to] = min(f[u], e.cap);
                        pq.emplace(nd, e.to);
                    }
```

```
                }
            }
            if (d[t] == INF) break;
            rep(i,0,n) if (d[i] < INF) p[i] += d[i];
            ll x = f[t];
            fl += x, cost += x * p[t];
            for (int v = t; v != s; v = pv[v]) {
                auto &e = g[pv[v]][pe[v]];
                e.cap -= x, g[v][e.rev].cap += x;
            }
        }
        return {fl, cost};
    } // e0d612
};
```

Matching bipartido com menor custo

Algoritmo Húngaro para assignment mínimo. Interface: passe uma matriz `a` de custo `n x m` com `n <= m`; `hungarian(a)` retorna `{cost, match}`, onde `match[i]` é a coluna escolhida para a linha `i`. Para maximização, negue os custos ou subtraia de uma constante grande. Complexidade: $O(n^2m)$.

c8c7c0 - graph/hungarian.cpp - 33 lines

```
pair<ll, vi> hungarian(vector<vector<ll>> a) {
    int n = sz(a), m = sz(a[0]);
    const ll INF = 4e18;
    vector<ll> u(n + 1), v(m + 1), minv(m + 1);
    vi p(m + 1), way(m + 1), ans(n);
    rep(i, 1, n + 1) {
        p[0] = i;
        int j0 = 0;
        fill(all(minv), INF);
        vector<char> used(m + 1);
        do {
            used[j0] = 1;
            int i0 = p[j0], j1 = 0;
            ll delta = INF;
            rep(j, 1, m + 1) if (!used[j]) {
                ll cur = a[i0 - 1][j - 1] - u[i0] - v[j];
                if (cur < minv[j]) minv[j] = cur, way[j] = j0;
                if (minv[j] < delta) delta = minv[j], j1 = j;
            }
            rep(j, 0, m + 1)
                if (used[j]) u[p[j]] += delta, v[j] -= delta;
                else minv[j] -= delta;
            j0 = j1;
        } while (p[j0]);
        do {
            int j1 = way[j0];
            p[j0] = p[j1];
            j0 = j1;
        } while (j0);
    }
    rep(j, 1, m + 1) if (p[j]) ans[p[j] - 1] = j - 1;
    return {-v[0], ans};
} // c8c7c0
```

Árvores

Base

Base de árvore com parentesco, profundidade, tamanhos de subárvore e heavy-light order. Interface: construa `Tree t(n)`, adicione arestas com `addEdge(u, v)` e finalize com `build(root)`. Depois disso, campos como `par`, `dep`, `head`, `in`, `out` e `rev` ficam prontos para uso. Complexidade: $O(n)$.

4c7dc0 - tree/base.cpp - 37 lines

```
struct Tree {
    int n, timer = 0;
    vector<vi> g;
    vi par, dep, cnt, head, in, out, rev;
    Tree(int n = 0)
        : n(n), g(n), par(n, -1), dep(n), cnt(n),
          head(n), in(n), out(n), rev(n) {}
    void addEdge(int u, int v) {
        g[u].pb(v);
        g[v].pb(u);
    } // 70fc01
    void dfsSz(int v, int p = -1) {
        par[v] = p;
        cnt[v] = 1;
        if (p != -1) g[v].erase(find(all(g[v]), p));
        for (int &u : g[v]) {
            dep[u] = dep[v] + 1;
            dfsSz(u, v);
```

```
            cnt[v] += cnt[u];
            if (cnt[u] > cnt[g[v][0]]) swap(u, g[v][0]);
        }
    } // 78db39
    void dfsHld(int v) {
        in[v] = timer;
        rev[timer++] = v;
        for (int u : g[v]) {
            head[u] = (u == g[v][0] ? head[v] : u);
            dfsHld(u);
        }
        out[v] = timer;
    } // 53c04a
    void build(int root = 0) {
        dfsSz(root);
        head[root] = root;
        dfsHld(root);
    } // 4540b8
};
```

Utilidades

Utilitários que assumem uma `Tree` já construída. Interface:

- `isAnc(t, a, b)` testa se `a` é ancestral de `b`.
- `subtree(t, v)` devolve o intervalo Euler da subárvore.
- `lca(t, a, b)` e `dist(t, a, b)` fazem LCA e distância.
- `kthAnc(t, v, k)` sobe `k` ancestrais.
- `jump(t, a, b, k)` pega o k-ésimo vértice no caminho `a -> b`.
- `getPath(t, a, b, f)` percorre o caminho em segmentos Euler e chama `f(l, r)`.

Complexidade: `isAnc` e `subtree` são $O(1)$; `lca`, `dist`, `kthAnc` e `jump` são $O(\log n)$; `getPath` enumera $O(\log n)$ segmentos.

178120 - tree/helpers.cpp - 43 lines

```
bool isAnc(Tree& t, int a, int b) {
    return t.in[a] <= t.in[b] && t.out[b] <= t.out[a];
} // eb6def
pii subtree(Tree& t, int v) {
    return {t.in[v], t.out[v]};
} // 15c9eb
int lca(Tree& t, int a, int b) {
    while (t.head[a] != t.head[b]) {
        if (t.dep[t.head[a]] > t.dep[t.head[b]]) a = t.par[t.head[a]];
        else b = t.par[t.head[b]];
    }
    return t.dep[a] < t.dep[b] ? a : b;
} // ba5b42
int dist(Tree& t, int a, int b) {
    int c = lca(t, a, b);
    return t.dep[a] + t.dep[b] - 2 * t.dep[c];
} // e0c63e
int kthAnc(Tree& t, int v, int k) {
    while (v != -1) {
        int h = t.head[v], d = t.dep[v] - t.dep[h];
        if (k <= d) return t.rev[t.in[v] - k];
        k -= d + 1;
        v = t.par[h];
    }
    return -1;
} // f240ac
// k-th vertex on path a -> b, 0-indexed
int jump(Tree& t, int a, int b, int k) {
    int c = lca(t, a, b);
    int da = t.dep[a] - t.dep[c], db = t.dep[b] - t.dep[c];
    if (k < 0 || k > da + db) return -1;
    if (k <= da) return kthAnc(t, a, k);
    return kthAnc(t, b, da + db - k);
} // b9e7b6
void getPath(Tree& t, int a, int b, auto f) {
    while (t.head[a] != t.head[b]) {
        if (t.dep[t.head[a]] > t.dep[t.head[b]]) swap(a, b);
        f(t.in[t.head[b]], t.in[b] + 1);
        b = t.par[t.head[b]];
    }
    if (t.dep[a] > t.dep[b]) swap(a, b);
    f(t.in[a], t.in[b] + 1);
} // 6ee927
```

LCA

LCA por Euler tour + sparse table. Interface: construa `LCA lca(g, root)` a partir da lista de adjacência e consulte pares com `lca.lca(a, b)`. Preprocessamento $O(n \log n)$, consulta $O(1)$.

1d44d9 - tree/lca.cpp - 23 lines

```
using RMQ = SparseTable<int, [](int a, int b) { return min(a, b); }>;

struct LCA {
    int t = 0;
    vi tm, path, ret;
    RMQ rmq;
    LCA(vector<vi> &g, int r = 0) : tm(g.size()) {
        dfs(g, r, -1);
        rmq.build(ret);
    }
    void dfs(vector<vi> &g, int x, int pre) {
        tm[x] = t++;
        for (int y: g[x]) if (y != pre) {
            path.pb(x), ret.pb(tm[x]);
            dfs(g, y, x);
        }
    } // 2e7ca5
    int lca(int x, int y) {
        if (x == y) return x;
        tie(x, y) = minmax(tm[x], tm[y]);
        return path[rmq.query(x, y)];
    } // e8105a
};
```

Matching em Árvores

Emparelhamento máximo em árvore por DFS gulosa em pós-ordem. Interface: passe a lista de adjacência 0-indexada e a raiz opcional; o retorno é um vetor `match` com o parceiro de cada vértice, ou `-1` se ele ficar livre. Complexidade: $O(n)$.

6e0886 - tree/tree-matching.cpp - 17 lines

```
vi treeMatching(vector<vi> &g, int root = 0) {
    int n = sz(g);
    vi par(n, -1), ord{root}, mat(n, -1);
    rep(i,0,sz(ord)) {
        int v = ord[i];
        for (int u : g[v]) if (u != par[v]) {
            par[u] = v;
            ord.pb(u);
        }
    }
    for (int i = n - 1; i > 0; --i) {
        int v = ord[i], p = par[v];
        if (mat[v] == -1 && mat[p] == -1)
            mat[v] = p, mat[p] = v;
    }
    return mat;
} // 6e0886
```

Strings

KMP

KMP/prefix function. Interface: chame `kmp(s)`; o retorno é o vetor `pre`, onde `pre[i]` é o tamanho do maior prefixo próprio de `s` que também é sufixo de `s[0..i]`. Útil para matching, periodicidade e automatos simples. Complexidade: $O(n)$.

cb52b3 - strings/kmp.cpp - 10 lines

```
vi kmp(string s) {
    int n = sz(s), len = 0;
    vi pre(n);
    rep (i, 1, n) {
        while (len > 0 && s[i] != s[len]) len = pre[len - 1];
        if (s[i] == s[len]) len++;
        pre[i] = len;
    }
    return pre;
} // cb52b3
```

Z-function

Z-function linear. Interface: chame `zFunc(s)`; o retorno é `z`, onde `z[i]` é o maior comprimento tal que o prefixo de `s` casa com `s[i..]`. Para matching de padrão, monte `p + '#' + t` e procure posições com `z[i] = |p|`. Complexidade: $O(n)$.

fc69f3 - strings/z-function.cpp - 12 lines

```
vi zFunc(string s) {
    int n = sz(s), l = 0, r = 0;
    vi z(n);
    if (!n) return z;
    rep(i, 1, n) {
        if (i < r) z[i] = min(r - i, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
```

```
        if (i + z[i] > r) l = i, r = i + z[i];
    }
    z[0] = n;
    return z;
} // fc69f3
```

Array de sufixos

Suffix array por doubling. Interface: chame `sufArr(s)`; o retorno é um vetor `sa` em que `sa[i]` é o índice de início do i-ésimo sufixo na ordem lexicográfica. A implementação também funciona para sequências comparáveis, não só para `string`. Complexidade: $O(n \log n)$.

f69f29 - strings/suffix-array.cpp - 27 lines

```
vi sufArr(string &s) { // Could also be vector<T> &s
    int n = sz(s);
    vi c(n), d(n), e(n), sb(n), sa(n), cnt(n + 1);
    iota(all(sa), 0);
    sort(all(sa), [&](int i, int j) { return s[i] < s[j]; });
    c[sa[0]] = 1;
    for (int i = 1; i < n; i++)
        c[sa[i]] = c[sa[i - 1]] + (s[sa[i]] != s[sa[i - 1]]);
    for (int k = 1; c[sa[n - 1]] != n; k <<= 1) {
        rep (i, 0, n) d[i] = i + k < n ? c[i + k] : 0;
        auto srt = [&](auto &C) {
            fill(all(cnt), 0);
            rep (i, 0, n) cnt[C[i] + 1]++;
            rep (i, 0, n) cnt[i + 1] += cnt[i];
            for (int i: sa) sb[cnt[C[i]]++] = i;
            swap(sa, sb);
        };
        srt(d); srt(c);
        e[sa[0]] = 1;
        rep(i,1,n) {
            int a = sa[i-1], b = sa[i];
            e[b] = e[a] + (c[a] != c[b] || d[a] != d[b]);
        }
        swap(c, e);
    }
    return sa;
} // f69f29
```

Maior prefixo comum (LCP)

Algoritmo de Kasai. Interface: passe a string `s` e o suffix array `sa`; o retorno é um vetor `lcp` de tamanho `n`, com `lcp[0] = 0`, onde `lcp[i]` é o maior prefixo comum entre `sa[i]` e `sa[i - 1]`. Complexidade: $O(n)$.

558ffb - strings/lcp.cpp - 12 lines

```
vi lcpArr(string &s, vi &sa) {
    int n = sz(s), k = 0;
    vi rnk(n), lcp(n);
    rep(i,0,n) rnk[sa[i]] = i;
    rep(i,0,n) if (rnk[i]) {
        int j = sa[rnk[i] - 1];
        while (i + k < n && j + k < n && s[i + k] == s[j + k]) k++;
        lcp[rnk[i]] = k;
        if (k) k--;
    }
    return lcp;
} // 558ffb
```

Árvore de sufixos

Construção linear da árvore de sufixos a partir do suffix array. Interface: monte `ST st(s, sa, lcp)` usando a string, o suffix array e o vetor de LCP no formato deste notebook. Os nós ficam em `st.t`; cada nó guarda intervalo `[l, r)`, pai `p`, filhos `ch` e, nas folhas, `suf` com o índice do sufixo correspondente. O vetor `st.dep` guarda a profundidade em caracteres de cada nó. Complexidade: $O(n)$.

3e9f4e - strings/suffix-tree.cpp - 31 lines

```
struct ST {
    struct N {
        int l, r, p, suf;
        vi ch;
    };
    vector<N> t;
    vi dep;
    int nn(int l, int r, int p, int suf = -1) {
        t.pb({l, r, p, suf, {}});
        dep.pb(p == -1 ? 0 : dep[p] + r - l);
        if (p != -1) t[p].ch.pb(sz(t) - 1);
        return sz(t) - 1;
    } // aa5bfe
```

```
    ST(string& s, vi& sa, vi& lcp) {
        int n = sz(s), rt = nn(0, 0, -1), last = rt;
        rep(i,0,n) {
            int x = sa[i], k = i ? lcp[i] : 0;
            while (dep[last] > k) last = t[last].p;
            if (dep[last] < k) {
                int y = t[last].ch.back(), z = nn(t[y].l, t[y].l + k - dep[last],
                ↪  last);
                t[last].ch.back() = z;
                t[y].p = z;
                t[y].l += k - dep[last];
                t[z].ch.pb(y);
                dep[z] = k;
                last = z;
            }
            last = nn(x + k, n, last, x);
        }
    }
};
```

Automato de sufixos

Automato de sufixos para alfabeto `a-z`. Interface: crie `SAM sam` e chame `extend(c)` para cada caractere da string. Os estados ficam em `sam.t`; cada nó guarda `len`, `link`, transições `to` e `cnt`. A função `ord()` devolve os estados em ordem crescente de `len`, útil para propagar DP ou contagens pelos suffix links. Complexidade: $O(n)$ para construir o automato e $O(|SAM|)$ para iterar `ord()`.

a67d77 - strings/suffix-automaton.cpp - 40 lines

```
struct SAM {
    struct N {
        int len = 0, link = -1;
        array<int, 26> to;
        ll cnt = 0; // endpos count after calcCnt()
        N() { to.fill(-1); }
    };
    vector<N> t{{}};
    int last = 0;
    void extend(char c) {
        int x = c - 'a', cur = sz(t), p = last;
        t.eb();
        t[cur].len = t[last].len + 1;
        t[cur].cnt = 1;
        for (; p != -1 && t[p].to[x] == -1; p = t[p].link) t[p].to[x] = cur;
        if (p == -1) t[cur].link = 0;
        else {
            int q = t[p].to[x];
            if (t[p].len + 1 == t[q].len) t[cur].link = q;
            else {
                int cl = sz(t);
                t.pb(t[q]);
                t[cl].len = t[p].len + 1;
                t[cl].cnt = 0;
                for (; p != -1 && t[p].to[x] == q; p = t[p].link) t[p].to[x] = cl;
                t[q].link = t[cur].link = cl;
            }
        }
        last = cur;
    } // 1f27c6
    vi ord() {
        int mx = 0;
        for (auto &u : t) mx = max(mx, u.len);
        vi cnt(mx + 1), ord(sz(t));
        for (auto &u : t) cnt[u.len]++;
        rep(i,1,mx+1) cnt[i] += cnt[i - 1];
        for (int i = sz(t) - 1; i >= 0; i--) ord[--cnt[t[i].len]] = i;
        return ord;
    } // a8d426
};
```

Para obter o número de substrings distintas, basta calcular a soma de `t[i].len - t[t[i].link].len` por todo o automato. Para obter o tamanho dos conjuntos `endpos` de cada estado, basta propagar os `cnt` por suffix links. Lembre-se que o automato de sufixos é um `DAG`.

Encontrando os palíndromos máximos em uma string

Algoritmo de Manacher. Interface: use `auto [d1, d2] = manacher(s)`. `d1[i]` é o raio do maior palíndromo ímpar centrado em `i`; `d2[i]` é o raio do maior palíndromo par entre `i - 1` e `i`. Os comprimentos são `2 * d1[i] - 1` e `2 * d2[i]`. Complexidade: $O(n)$.

59be6a - strings/manacher.cpp - 17 lines

```
pair<vi, vi> manacher(string& s) {
    int n = sz(s);
    vi d1(n), d2(n);
```

```
    for (int i = 0, l = 0, r = -1; i < n; i++) {
        int k = i > r ? 1 : min(d1[l + r - i], r - i + 1);
        while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) k++;
        d1[i] = k--;
        if (i + k > r) l = i - k, r = i + k;
    }
    for (int i = 0, l = 0, r = -1; i < n; i++) {
        int k = i > r ? 0 : min(d2[l + r - i + 1], r - i + 1);
        while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k]) k++;
        d2[i] = k--;
        if (i + k > r) l = i - k - 1, r = i + k;
    }
    return {d1, d2}; // {odd, even}
} // 59be6a
```

Aho-Corasick

Automato de Aho-Corasick para strings minúsculas. Interface: crie `AC ac`, chame `ac.add(pat)` para cada padrão e guarde o nó terminal retornado, depois execute `ac.build()`. Na consulta, avance com `v = ac.next(v, c)`; os links de falha ficam em `ac.t[v].link` e `ac.ord` permite propagar informação na árvore de sufixos em ordem reversa. Complexidade: $O(\sum |pat|)$ para inserir, $O(|AC| \cdot A)$ para `build()` com alfabeto fixo de tamanho $A = 26$, e $O(|texto|)$ para percorrer o texto.

05f5f7 - strings/aho-corasick.cpp - 38 lines

```
struct AC {
    static constexpr int A = 26;
    struct N {
        array<int, A> to{};
        int link = 0;
        N() { to.fill(-1); }
    };
    vector<N> t{{}};
    vi ord;
    int add(string& s) {
        int v = 0;
        for (char c : s) {
            int x = c - 'a';
            if (t[v].to[x] == -1) t[v].to[x] = sz(t), t.eb();
            v = t[v].to[x];
        }
        return v;
    } // bd83ba
    void build() {
        queue<int> q;
        rep(c,0,A) {
            int u = t[0].to[c];
            if (u == -1) t[0].to[c] = 0;
            else q.push(u);
        }
        ord.clear();
        while (sz(q)) {
            int v = q.front(); q.pop();
            ord.pb(v);
            rep(c,0,A) {
                int& u = t[v].to[c];
                if (u == -1) u = t[t[v].link].to[c];
                else t[u].link = t[t[v].link].to[c], q.push(u);
            }
        }
    } // 5e19d4
    int next(int v, char c) { return t[v].to[c - 'a']; }
};
```

Geometria

Base

Base inteira do template de geometria. Interface: inclua este snippet quando quiser um prelude comum com `Pt = complex<ll>`, os atalhos `xx/yy`, as operações `dot`, `cross`, `perp` e o comparador `sgn`. Vários snippets abaixo usam essa base em `ll`; os que fazem divisões ou interseções geométricas foram escritos em `ld` e precisam de `Pt = complex<ld>` em vez desta base inteira.

673f55 - geometry/core.cpp - 9 lines

```
using Pt = complex<ll>;
#define xx real()
#define yy imag()

ll dot(Pt a, Pt b) { return (conj(a) * b).xx; }
ll cross(Pt a, Pt b) { return (conj(a) * b).yy; }
Pt perp(Pt a) { return Pt(-a.yy, a.xx); }
const ld EPS = 1e-9;
int sgn(ld x) { return (x > EPS) - (x < -EPS); }
```

Interseção de retas

Este snippet assume `ld`. Interface: passe dois pontos de cada reta, `a,b` e `c,d`; o retorno é o ponto de interseção das retas infinitas `ab` e `cd`. Não é teste de segmentos. Assuma que as retas não são paralelas nem coincidentes, ou trate esse caso antes da chamada. Complexidade: $O(1)$.

2fc473 - geometry/line-intersection.cpp - 3 lines

```
Pt lineInter(Pt a, Pt b, Pt c, Pt d) {
    return a + (b - a) * cross(c - a, d - c) / cross(b - a, d - c);
} // 2fc473
```

Interseção de segmentos

Este snippet assume `ld` e comparações com `EPS`. Interface: passe os quatro endpoints dos segmentos. `segInter` retorna se os segmentos se intersectam, contando borda e colinearidade; `properInter` testa apenas interseção estrita. O retorno é só booleano, não o ponto de interseção. Se for usar coordenadas inteiras exatas, ajuste `EPS` como indicado no arquivo. Complexidade: $O(1)$.

f6de72 - geometry/segment-intersection.cpp - 16 lines

```
bool segInter(Pt a, Pt b, Pt c, Pt d) {
    int ab_c = sgn(cross(b - a, c - a)), ab_d = sgn(cross(b - a, d - a));
    int cd_a = sgn(cross(d - c, a - c)), cd_b = sgn(cross(d - c, b - c));
    if (ab_c * ab_d < 0 && cd_a * cd_b < 0) return 1;
    if (!ab_c && sgn(dot(c - a, c - b)) <= 0) return 1;
    if (!ab_d && sgn(dot(d - a, d - b)) <= 0) return 1;
    if (!cd_a && sgn(dot(a - c, a - d)) <= 0) return 1;
    if (!cd_b && sgn(dot(b - c, b - d)) <= 0) return 1;
    return 0;
} // 4f106a

bool properInter(Pt a, Pt b, Pt c, Pt d) {
    int ab_c = sgn(cross(b - a, c - a)), ab_d = sgn(cross(b - a, d - a));
    int cd_a = sgn(cross(d - c, a - c)), cd_b = sgn(cross(d - c, b - c));
    return ab_c * ab_d < 0 && cd_a * cd_b < 0;
} // 66a13a
```

Área de um polígono

Interface: passe os vértices do polígono em ordem circular, sem repetir o primeiro no fim. O retorno é a soma orientada das arestas, isto é, duas vezes a área com sinal; aplique `abs` e divida por 2 se quiser a área usual. Esta versão usa `ll`; o próprio arquivo indica as trocas necessárias para usar `ld`. Complexidade: $O(n)$.

b23e06 - geometry/polygon-area.cpp - 5 lines

```
ll polyArea(vector<Pt> &pt) { // ld: change ret type to ld
    ll n = sz(pt), s = 0; // ld: change to ld s
    rep (i, 0, n) s += cross(pt[i], pt[(i + 1) % n]);
    return s;
} // b23e06
```

Verificar se um ponto está dentro de um polígono

Apesar do nome do arquivo, este snippet é para polígono convexo. Interface: chame `pointInPoly(pt, q, border)`; `pt` deve ser convexo e estar em ordem anti-horária, `q` é o ponto consultado e `border = false` exclui a borda. Esta versão usa `ll`; o comentário no arquivo mostra como adaptar para `ld` com `EPS`. Complexidade: $O(\log n)$.

1ce015 - geometry/point-in-polygon.cpp - 10 lines

```
bool pointInPoly(vector<Pt> &pt, Pt q, bool border = true) {
    int n = pt.size(), t = border, l = 1, r = n - 1; // ld: ld t = border ? EPS : 0
    auto v = [&](int i) { return cross(q - pt[0], pt[i] - pt[0]); };
    if (v(l) >= t || v(r) <= -t) return 0;
    while (r - l > 1) {
        int m = (l + r) / 2;
        (v(m) > 0 ? r : l) = m; // ld: use v(m) > EPS
    }
    return cross(pt[r] - pt[l], pt[l] - q) < t; // ld: use the ld t above
} // 1ce015
```

Convex hull

Interface: passe o vetor de pontos `pt`; o retorno é a lista de índices dos vértices do convex hull em sentido anti-horário, referenciando o vetor original. Por padrão, pontos colineares sobre as arestas não entram no hull; o comentário no snippet mostra o que trocar se quiser incluí-los. Esta versão usa `ll`; para `ld`, troque as comparações como indicado no arquivo. Complexidade: $O(n \log n)$.

b43fd9 - geometry/convex-hull.cpp - 22 lines

```
vi convHull(vector<Pt> &pt) {
    int n = sz(pt), m;
    vi h, ord(n);
    auto add = [&]() {
        vi st;
        for (int i: ord) {
            while ((m = sz(st)) > 1) {
                Pt a = pt[st[m - 1]], b = pt[st[m - 2]], c = pt[i];
                if (cross(b - a, c - a) < 0) break; // 1) > for clockwise, <= to
                ↪  include non-vertices
                st.pop_back();
            }
            st.pb(i);
        }
        st.pop_back();
        h.insert(h.end(), all(st));
    };
    iota(all(ord), 0);
    auto top = [](auto a) { return pair(a.xx, a.yy); };
    sort(all(ord), [&](int i, int j) { return top(pt[i]) > top(pt[j]); });
    add(), reverse(all(ord)), add();
    return h;
} // b43fd9 // 1) ld: use sgn(cross(...)) < 0
```

Menor distância euclideana

Interface: passe o vetor de pontos e receba `{i, j}`, onde `i` e `j` são índices de um par mais próximo no vetor original. O snippet trabalha com distância quadrada internamente. Esta versão está preparada para `ll`; o próprio arquivo marca as trocas necessárias para usar `ld`. Complexidade: $O(n \log n)$.

264831 - geometry/closest-pair.cpp - 33 lines

```
ll dist2(Pt a, Pt b) { // ld: ll -> ld
    ll dx = a.xx - b.xx, dy = a.yy - b.yy; // ld: ll -> ld
    return dx * dx + dy * dy;
} // ed4927

pii closestPair(vector<Pt> p) {
    vector<pair<Pt,int>> a;
    rep(i,0,sz(p)) a.eb(p[i], i);
    sort(all(a), [](auto a, auto b) {
        return a.fi.xx != b.fi.xx ? a.fi.xx < b.fi.xx : a.fi.yy < b.fi.yy;
    });
    ll ans = dist2(a[0].fi, a[1].fi), l = 0;
    pii best = {a[0].se, a[1].se};
    set<tuple<ll,ll,int>> s; // ld: use tuple<ld, ld, int>
    s.insert({a[0].fi.yy, a[0].fi.xx, a[0].se});
    s.insert({a[1].fi.yy, a[1].fi.xx, a[1].se});
    rep (i, 2, sz(a)) {
        ll d = (ll)sqrtl((ld)ans) + 1; // ld: ld d = sqrtl(ans) + EPS
        while (a[i].fi.xx - a[l].fi.xx > d) {
            s.erase({a[l].fi.yy, a[l].fi.xx, a[l].se});
            l++;
        }
        auto it1 = s.lower_bound({a[i].fi.yy - d, LLONG_MIN, INT_MIN});
        auto it2 = s.upper_bound({a[i].fi.yy + d, LLONG_MAX, INT_MAX});
        for (auto it = it1; it != it2; ++it) {
            int j = get<2>(*it);
            ll cur = dist2(a[i].fi, p[j]); // ld: ld cur = ...
            if (cur < ans) ans = cur, best = {a[i].se, j};
        }
        s.insert({a[i].fi.yy, a[i].fi.xx, a[i].se});
    }
    return best;
} // 130018
```

Interseção de círculo e reta

Este snippet assume `ld`. Interface: passe o círculo por centro `c` e raio `r`, e a reta por um ponto `a` e vetor direção não nulo `d`. O retorno é a lista dos pontos de interseção entre o círculo e a reta infinita; ela vem com tamanho 0, 1 ou 2. Complexidade: $O(1)$.

21e90b - geometry/circle-line.cpp - 8 lines

```
vector<Pt> circLineInter(Pt c, ld r, Pt a, Pt d) {
    Pt f = a + d * dot(c - a, d) / norm(d);
    ld h2 = r * r - norm(c - f);
    if (sgn(h2) < 0) return {};
    ld dt = sqrt(max((ld)0, h2)) / abs(d);
    if (!sgn(h2)) return {f};
    return {f - dt * d, f + dt * d};
} // 21e90b
```

Interseção de círculos

Este snippet assume `ld`. Interface: passe os dois círculos `(c1, r1)` e `(c2, r2)`; o retorno é a lista dos pontos de interseção, com tamanho 0, 1 ou 2. O caso degenerado de círculos coincidentes não é tratado separadamente; se isso puder acontecer, filtre antes da chamada. Complexidade: $O(1)$.

d3b1f8 - geometry/circle-circle.cpp - 12 lines

```
vector<Pt> circCircInter(Pt c1, ld r1, Pt c2, ld r2) {
    Pt d = c2 - c1;
    ld dist = abs(d);
    if (!sgn(dist)) return {};
    ld a = (norm(d) + r1 * r1 - r2 * r2) / (2 * dist);
    ld h2 = r1 * r1 - a * a;
    if (sgn(h2) < 0) return {};
    Pt mid = c1 + d * (a / dist);
    if (!sgn(h2)) return {mid};
    Pt p = perp(d) * (sqrt(max((ld)0, h2)) / dist);
    return {mid - p, mid + p};
} // d3b1f8
```

Tangentes comuns a dois círculos

Este snippet assume `ld`. Interface: o retorno é uma lista de pares `{p1, p2}`, com um ponto de tangência em cada círculo, um em `c1` e outro em `c2`. O snippet enumera tangentes externas e internas, com até 4 respostas. Casos degenerados como círculos coincidentes devem ser tratados fora; em casos tangentes pode ser necessário remover duplicatas se isso importar no problema. Complexidade: $O(1)$.

0d00d5 - geometry/circle-tangents.cpp - 14 lines

```
vector<pair<Pt, Pt>> circTangents(Pt c1, ld r1, Pt c2, ld r2) {
    vector<pair<Pt, Pt>> res;
    for (int s: {1, -1}) {
        ld r = s * r2, dr = r1 - r;
        Pt d = c2 - c1;
        ld l2 = norm(d), h2 = max((ld)0, l2 - dr * dr);
        if (sgn(l2 - dr * dr) < 0) continue;
        for (int t: {1, -1}) {
            Pt n = (dr * d + perp(d) * (t * sqrt(h2))) / l2;
            res.pb({c1 + r1 * n, c2 + r * n});
        }
    }
    return res;
} // 0d00d5
```

Soma Minkowski de polígonos convexos

Interface: passe dois polígonos convexos em sentido anti-horário, sem repetir o primeiro vértice no fim; o retorno é o polígono $a \oplus b$, também em sentido anti-horário. A rotina pode manter vértices redundantes se houver arestas colineares; normalize depois se precisar. Esta versão usa `ll`; o comentário no arquivo mostra o que ajustar para `ld`. Complexidade: $O(|a| + |b|)$.

107f6b - geometry/minkowski.cpp - 17 lines

```
bool ptLt(Pt a, Pt b) { return a.xx != b.xx ? a.xx < b.xx : a.yy < b.yy; } // 1)
vector<Pt> minkowski(vector<Pt> a, vector<Pt> b) {
    auto norm = [&](vector<Pt>& p) {
        rotate(p.begin(), min_element(all(p), ptLt), p.end());
        p.pb(p[0]), p.pb(p[1]);
    };
    int n = sz(a), m = sz(b);
    norm(a), norm(b);
    vector<Pt> c;
    for (int i = 0, j = 0; i < n || j < m; ) {
        c.pb(a[i] + b[j]);
        ll z = cross(a[i + 1] - a[i], b[j + 1] - b[j]); // ld: keep z in ld
        if (z >= 0 && i < n) i++; // ld: use z >= -EPS
        if (z <= 0 && j < m) j++; // ld: use z <= EPS
    }
    return c;
} // f04989 // 1) ld: a.xx != b.xx -> sgn(a.xx - b.xx)
```

Corte de Polígono

Este snippet assume `ld`. Interface: chame `polyCut(p, a, b)`; o retorno é o polígono `p` cortado pelo semiplano à esquerda da reta direcionada $a \to b$. A reta é infinita e o resultado sai em ordem circular. Útil para encadear cortes por semiplanos ou clipar por um polígono convexo. Complexidade: $O(n)$.

a60a48 - geometry/polygon-cut.cpp - 10 lines

```
vector<Pt> polyCut(vector<Pt> p, Pt a, Pt b) {
    vector<Pt> q;
    rep(i,0,sz(p)) {
```

```
        Pt c = p[i], d = p[(i + 1) % sz(p)];
        int sc = sgn(cross(b - a, c - a)), sd = sgn(cross(b - a, d - a));
        if (sc >= 0) q.pb(c);
        if (sc * sd < 0) q.pb(lineInter(a, b, c, d));
    }
    return q;
} // a60a48
```

Fórmulas, Teoremas e Fatos

Combinatória e Sequências

Progressão aritmética.

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

$$a + (a+d) + \cdots + (a + (n-1)d) = \frac{n(2a + (n-1)d)}{2}$$

Progressão geométrica.

$$1 + r + r^2 + \cdots + r^n = \frac{r^{n+1} - 1}{r - 1} \qquad (r \neq 1)$$

$$a + ar + \cdots + ar^{n-1} = a\frac{r^n - 1}{r - 1} \qquad (r \neq 1)$$

Somas de potências comuns.

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{i=1}^{n} i^3 = \left(\frac{n(n+1)}{2}\right)^2$$

Coeficiente binomial.

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

Coeficiente de multiconjunto.

$$\left(\!\binom{n}{k}\!\right) = \binom{n+k-1}{k}$$

Número de formas de escolher k elementos entre n tipos, com repetição.

Estrelas e Barras. O número de soluções inteiras não negativas de

$$x_1 + \cdots + x_k = n$$

é

$$\binom{n+k-1}{k-1}.$$

O número de soluções inteiras positivas é

$$\binom{n-1}{k-1}.$$

Estrelas e barras com cotas inferiores. O número de soluções inteiras de

$$x_1 + \cdots + x_n = S, \qquad x_i \geq l_i$$

é

$$\binom{S - \sum l_i + n - 1}{n-1},$$

após a substituição $y_i = x_i - l_i$.

Número de Catalan.

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1}$$

Lembrete de interpretações de Catalan.

$$C_n$$

conta sequências balanceadas de parênteses de tamanho $2n$, formas de árvores binárias enraizadas com n nós internos, triangulações de um $(n+2)$-gono e matchings perfeitos sem cruzamento em $2n$ pontos.

Desarranjos.

$$D_n = n! \sum_{k=0}^{n} \frac{(-1)^k}{k!}$$

e

$$D_n = (n-1)(D_{n-1} + D_{n-2})$$

Convolução binomial (Vandermonde).

$$\sum_k \binom{a}{k}\binom{b}{n-k} = \binom{a+b}{n}.$$

Teorema do Casamento de Hall. Um grafo bipartido (L, R) tem um matching saturando L sse

$$\forall S \subseteq L, \quad |N(S)| \geq |S|.$$

Teorema de König. Em um grafo bipartido,

$$\text{matching máximo} = \text{cobertura mínima}.$$

Teorema de Dilworth. Em qualquer poset finito, o tamanho máximo de uma anticadeia é igual ao número mínimo de cadeias necessárias para cobrir todos os elementos.

Teorema de Mirsky. Em qualquer poset finito, o tamanho máximo de uma cadeia é igual ao número mínimo de anticadeias necessárias para cobrir todos os elementos.

Erdős–Szekeres. Toda sequência de $(r-1)(s-1)+1$ números distintos contém ou uma subsequência crescente de tamanho r, ou uma subsequência decrescente de tamanho s.

Números de Stirling do segundo tipo.

$$S(n,k) = S(n-1,k-1) + k\,S(n-1,k)$$

Forma fechada:

$$S(n,k) = \frac{1}{k!}\sum_{i=0}^{k}(-1)^i\binom{k}{i}(k-i)^n$$

Números de Stirling do primeiro tipo (sem sinal).

$$c(n,k) = c(n-1,k-1) + (n-1)c(n-1,k)$$

Números de Bell.

$$B_n = \sum_{k=0}^{n} S(n,k)$$

Triangulações de polígono convexo. Número de triangulações de um n-gono:

$$C_{n-2} = \frac{1}{n-1}\binom{2n-4}{n-2}$$

Valor esperado por indicadores. Se

$$X = \sum_i I_i,$$

então

$$\mathbb{E}[X] = \sum_i \Pr(I_i = 1).$$

Contagem de iterações em submasks. Número total de pares (m, s) com $s \subseteq m \subseteq \{0, \ldots, n-1\}$:

$$3^n.$$

Enumeração de Pólya. Burnside mais decomposição em ciclos para contar colorações sob simetria. Em geral, só vale a pena deixar como lembrete se o time realmente usa.

Teoria dos Números

Truques de floor/ceiling.

$$\left\lceil \frac{a}{b} \right\rceil = \left\lfloor \frac{a+b-1}{b} \right\rfloor \qquad (a, b > 0)$$

$$\left\lceil \frac{a}{b} \right\rceil = \frac{a+b-1}{b}$$

em aritmética inteira, para $a, b > 0$.

Contagem / soma de divisores. Se

$$n = \prod p_i^{e_i},$$

então

$$d(n) = \prod (e_i + 1),$$

$$\sigma(n) = \prod \frac{p_i^{e_i+1} - 1}{p_i - 1}.$$

Phi de Euler.

$$\varphi(n) = n \prod_{p|n} \left(1 - \frac{1}{p}\right)$$

Fórmula de Legendre.

$$v_p(n!) = \sum_{k \geq 1} \left\lfloor \frac{n}{p^k} \right\rfloor$$

Teorema de Lucas. Para primo p, se

$$n = \sum n_i p^i, \qquad k = \sum k_i p^i,$$

então

$$\binom{n}{k} \equiv \prod_i \binom{n_i}{k_i} \pmod{p}.$$

Teorema de Kummer. O expoente de um primo p em $\binom{n}{k}$ é igual ao número de vai-uns ao somar k e $n-k$ na base p.

LTE (versão comum). Se p é um primo ímpar, $p \mid (a-b)$ e $p \nmid ab$, então

$$v_p(a^n - b^n) = v_p(a-b) + v_p(n).$$

Além disso, para n ímpar e $p \mid (a+b)$,

$$v_p(a^n + b^n) = v_p(a+b) + v_p(n).$$

Condição geral de compatibilidade do CRT. O sistema

$$x \equiv a_i \pmod{m_i}$$

tem solução sse, para todos i, j,

$$a_i \equiv a_j \pmod{\gcd(m_i, m_j)}.$$

Quando tem solução, ela é única módulo $\operatorname{lcm}(m_1, \ldots, m_k)$.

Inversão de Möbius. Se

$$g(n) = \sum_{d|n} f(d),$$

então

$$f(n) = \sum_{d|n} \mu(d)\, g\left(\frac{n}{d}\right).$$

Truque de agrupar quocientes. O valor de

$$\left\lfloor \frac{n}{i} \right\rfloor$$

muda apenas $O(\sqrt{n})$ vezes quando i varia. Mais precisamente, todos os i com o mesmo quociente formam um intervalo

$$i \in \left[\left\lfloor \frac{n}{q+1} \right\rfloor + 1,\ \left\lfloor \frac{n}{q} \right\rfloor\right].$$

Existência de raiz primitiva. Existem raízes primitivas módulo n sse

$$n \in \{1, 2, 4, p^k, 2p^k\},$$

onde p é um primo ímpar.

Teorema de Wilson. Para primo p,

$$(p-1)! \equiv -1 \pmod{p}.$$

Grafos e Jogos

Condições para trilha/circuito euleriano (direcionado). Ignorando os vértices isolados, todos os vértices relevantes devem pertencer a uma única componente fracamente conexa. Então:

- existe circuito euleriano sse $\deg^+(v) = \deg^-(v)$ para todo vértice;
- existe trilha euleriana sse exatamente um vértice tem $\deg^+(v) = \deg^-(v) + 1$, exatamente um tem $\deg^-(v) = \deg^+(v) + 1$, e todos os outros são balanceados.

Condições para trilha/circuito euleriano (não direcionado). Ignorando os vértices isolados, todos os vértices relevantes devem pertencer a uma única componente conexa. Então:

- existe circuito euleriano sse todo vértice tem grau par;
- existe trilha euleriana sse exatamente 0 ou 2 vértices têm grau ímpar.

Critério de 2-SAT. Uma instância de 2-SAT é satisfatível sse, para toda variável x, os literais x e $\neg x$ estão em SCCs diferentes do grafo de implicação.

Restrições de diferença $\leftrightarrow$ menores caminhos. Um sistema de desigualdades da forma

$$x_v - x_u \leq w$$

pode ser modelado por uma aresta $u \to v$ de peso w. Então:

- o sistema é viável sse não existe ciclo negativo;
- uma solução viável é dada pelas distâncias a partir de uma super-fonte.

Cobertura por caminhos em DAG via matching. O número mínimo de caminhos disjuntos em vértices que cobrem todos os vértices de um DAG é

$$n - (\text{matching máximo na expansão bipartida do DAG}).$$

Teorema Matrix-Tree de Kirchhoff. O número de árvores geradoras de um grafo é igual a qualquer cofator de sua matriz Laplaciana.
Teorema de Sprague–Grundy. Todo jogo imparcial finito em normal play é equivalente a uma pilha de Nim cujo tamanho é seu número de Grundy; o número de Grundy é o mex dos números de Grundy das jogadas possíveis. O número de Grundy de uma soma disjunta é o xor dos números de Grundy das componentes.

Intervalos, Prefixos e Contribuições

Mediana minimiza soma de desvios absolutos.

$$\arg\min_x \sum_i |x - a_i|$$

é qualquer mediana dos a_i. Mais geralmente,

$$\arg\min_x \sum_i w_i |x - a_i|$$

é qualquer mediana ponderada.
Média minimiza soma de quadrados.

$$\arg\min_x \sum_i (x - a_i)^2$$

é

$$x = \frac{1}{n} \sum_i a_i.$$

Se x precisa ser inteiro, teste $\lfloor \bar{a} \rfloor$ e $\lceil \bar{a} \rceil$.
Fórmula de contribuição na soma de todos os subarrays.

$$\sum_{0 \le l \le r < n} \sum_{i=l}^{r} a_i = \sum_{i=0}^{n-1} a_i (i+1)(n-i).$$

O elemento a_i aparece em exatamente $(i+1)(n-i)$ subarrays.
Contribuição de uma aresta para a soma das distâncias na árvore. Se remover uma aresta de peso w separa a árvore em componentes de tamanhos s e $n-s$, então sua contribuição para

$$\sum_{u<v} \mathrm{dist}(u, v)$$

é

$$w \cdot s(n-s).$$

Máximo número de intervalos disjuntos. Ordenar os intervalos por extremo direito crescente e escolher gulosa-mente produz um conjunto máximo de intervalos dois a dois disjuntos.
Mínimo de pontos para perfurar intervalos. Ordenar os intervalos por extremo direito crescente e escolher repetidamente o extremo direito do primeiro intervalo ainda não coberto produz um conjunto mínimo de pontos que intersecta todos os intervalos.
Intervalos dois a dois intersectantes têm interseção comum. Para intervalos na reta, se todo par intersecta, então todos compartilham um ponto em comum. Isso falha em dimensões maiores, mas é muito útil em 1D.
Truque dos intervalos circulares. Em problemas sobre um círculo, cortar em um ponto escolhido transforma intervalos circulares em intervalos comuns. Muitas vezes isso se resolve duplicando o array/lista de ângulos e trabalhando numa estrutura linearizada de tamanho $2n$.

Strings

Teorema de Fine–Wilf. Se uma string tem períodos p e q, e seu tamanho é pelo menos

$$p + q - \gcd(p, q),$$

então ela também tem período $\gcd(p, q)$.
Hash polinomial de substring. Se

$$H[i+1] = H[i] \cdot B + s[i],$$

então

$$hash(l, r) = H[r] - H[l] \cdot B^{r-l}$$

para o intervalo semiaberto $[l, r)$.
Substrings distintas. Usando suffix array:

$$\#\text{substrings distintas} = \frac{n(n+1)}{2} - \sum lcp[i]$$

Usando suffix automaton:

$$\#\text{substrings distintas} = \sum_{v \neq root} (len(v) - len(link(v)))$$

Geometria

Transformações da distância Manhattan. Em 2D,

$$|x_1 - x_2| + |y_1 - y_2| = \max\Big(|(x+y)' - (x+y)|,\, |(x-y)' - (x-y)|\Big)$$

Na prática, para a maior distância entre pares:

$$\max_{i,j} |x_i - x_j| + |y_i - y_j| = \max(D_+, D_-)$$

onde

$$D_+ = \max(x+y) - \min(x+y), \qquad D_- = \max(x-y) - \min(x-y)$$

Teorema de Pick. Para um polígono simples na grid,

$$A = I + \frac{B}{2} - 1,$$

onde I é o número de pontos de grid internos e B é o número de pontos de grid na fronteira.

Pontos de grid em um segmento. O número de pontos de grid no segmento de (x_1, y_1) até (x_2, y_2) é

$$\gcd(|x_1 - x_2|, |y_1 - y_2|) + 1.$$

Distância de ponto a reta. Distância do ponto P à reta passando por A, B:

$$\frac{|\operatorname{cross}(B - A, P - A)|}{|B - A|}$$

Fórmula de Heron. Para lados a, b, c e semiperímetro

$$s = \frac{a+b+c}{2},$$

a área do triângulo é

$$\sqrt{s(s-a)(s-b)(s-c)}.$$

Lei dos cossenos.

$$c^2 = a^2 + b^2 - 2ab\cos C$$

Ângulo pelo produto escalar.

$$\cos\theta = \frac{a \cdot b}{|a||b|}$$

Raio da circunferência circunscrita e inscrita. Para área do triângulo Δ:

$$R = \frac{abc}{4\Delta}, \qquad r = \frac{\Delta}{s}$$

onde $s = \frac{a+b+c}{2}$.

Comprimento da corda. Para raio r e ângulo central θ:

$$2r\sin\frac{\theta}{2}$$

Teorema de Helly em $\mathbb{R}^2$. Para uma família finita de conjuntos convexos no plano, se todo subconjunto de 3 deles intersecta, então todos intersectam.

Teorema de Helly em $\mathbb{R}^d$. Para conjuntos convexos em $\mathbb{R}^d$, se todo subconjunto de $d+1$ deles intersecta, então todos intersectam.

Teorema de Carathéodory. Se um ponto pertence ao fecho convexo de um conjunto em $\mathbb{R}^d$, então ele pertence ao fecho convexo de no máximo $d+1$ pontos desse conjunto.

Teorema de Radon. Qualquer conjunto com pelo menos $d+2$ pontos em $\mathbb{R}^d$ pode ser particionado em dois subconjuntos disjuntos cujos fechos convexos se intersectam.

Calota esférica / fórmulas avançadas de geometria. Em geral não valem o espaço no notebook, a menos que sua região realmente cobre isso.

DP e Otimizações

Desigualdade de Monge. Um array A é Monge se

$$A[i][j] + A[i'][j'] \leq A[i][j'] + A[i'][j]$$

para $i \leq i'$ e $j \leq j'$. Essa é a condição estrutural por trás de várias otimizações de DP.

Condição da otimização de Knuth. Para DP de intervalos,

$$opt[l][r-1] \leq opt[l][r] \leq opt[l+1][r]$$

sob as hipóteses usuais de desigualdade do quadrilátero / monotonicidade.

Condição da otimização por divide and conquer. Se o índice minimizante é monótono,

$$opt[i][j] \leq opt[i][j+1],$$

então cada camada pode ser calculada com otimização por divide and conquer.

Ideias

Gerais

- Em problemas envolvendo somas em intervalos de arrays, pode ser mais fácil trabalhar com o array de somas de prefixos.
- Muitas DPs de bitmask podem ser simplificadas para usar apenas alguma propriedade da bitmask. (Número de bits, etc)
- Se o problema descreve um processo que parece complicado, talvez seja mais fácil ver o processo ao contrário.

Calculando somas e integrais

- Verificar se existe uma fórmula fechada
- Inverter a ordem da soma (em somatórios duplos)
- Transformar em somatório duplo e inverter a ordem
- Verificar se a soma pode ser calculada de um jeito eficiente (sem ser uma fórmula fechada)
- No caso de calcular várias somas, verificar se existe uma relação entre elas (Calcular uma pode ajudar a calcular outra)

Problemas de Otimização

- Encontrar um upper bound e provar que ele sempre é alcançável.
- Encontrar uma construção boa, e provar que não é possível fazer melhor.
- Escrever um brute-force e testar com casos pequenos para encontrar algum padrão.

Debugging

Geral

- Trocou nomes de variáveis? (m por n, j por i, ...)
- Esqueceu algum caso especial? ($n = 1$, $m = 0$, ...)
- Esqueceu de trocar entre 0-based e 1-based?

WA - Wrong Answer

- Esqueceu o `define int ll`?
- Esqueceu algum caso base?
- verificar se o valor inicial da DP está correto.

RE - Runtime Error

- Verificar asserts.
- Verificar limites de vectors.
- Verificar se `maxn` é o suficiente.